

A Project Report on
**Dynamic Group-Based Ping Lock Arbiter with Skip
Logic for 4 Requesters**

**Submitted in partial fulfilment of the requirements
for the award of the degree of**

Bachelor of Technology (B.Tech)

in

Electronics and Communication Engineering

Under the guidance of

Dr. Ch. V. M. S. N. Pavan Kumar M.Tech, Ph.D
Assistant Professor

Submitted by

M. Rupa Mani Latha	(Y22AEC509)
M. Pavan Krishna	(L23AEC599)
M. Kanchana	(Y22AEC514)
O. Manikanta	(Y22AEC529)



Department of Electronics and Communication Engineering

Bapatla Engineering College (Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA-522 101, A.P, INDIA

April 2026

Department of Electronics and Communication Engineering

Bapatla Engineering College (Autonomous)

(Affiliated to Acharya Nagarjuna University)



CERTIFICATED

This is to certify that the project entitled as “ **Dynamic Group Based Ping Lock Arbiter with Skip Logic For 4 Requesters**” is the Bonafide work of M. Rupa Mani Latha (Y22AEC509), M. Pavan Krishna (L23AEC599), M. Kanchana (Y22AEC514), O. Manikanta (Y22AEC529) submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (**B.Tech**) in Electronics and Communication Engineering (**ECE**) by Acharya Nagarjuna University during the academic year 2025-2026.

Guide:

Dr. Ch. V. M. S. N. Pavan Kumar, M.Tech, Ph.D
Assistant Professor,
Department of ECE,
Bapatla Engineering College,
Bapatla.

Head of the Department:

Dr. B. Chandra Mohan, M.Tech, Ph.D
Professor & HOD,
Department of ECE,
Bapatla Engineering College,
Bapatla.



Bapatla Engineering College, Bapatla (Autonomous)

Department of Electronics and Communications Engineering

VISION

- To produce globally competitive and socially responsible Electronics and Communication Engineering graduates to cater to the ever-changing needs of society.

MISSION

- To provide quality education in the domain of Electronics and Communication Engineering with advanced pedagogical methods.
- To provide self-learning capabilities to enhance employability and entrepreneurial skills and to inculcate human values and ethics to make learners sensitive to wards societal issues.
- To excel in the research and development activities related to Electronics and Communication Engineering.



Bapatla Engineering College, Bapatla (Autonomous)

Department of Electronics and Communication Engineering Program Outcomes (PO's)

Engineering Graduates will be able to:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.



Bapatla Engineering College, Bapatla (Autonomous)

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Department of Electronics and Communications Engineering

PROGRAM SPECIFIC OUTCOMES (PSO's)

A graduate of the Electronics and Communication Engineering Program will be able to:

- **PSO1:** Develop and implement modern Electronic Technologies using analytical methods to meet current as well as future industrial and societal needs
- **PSO2:** Analyze and develop VLSI, IoT and Embedded Systems for desired specifications to solve real world complex problems.
- **PSO3:** Apply machine learning and deep learning techniques in communication and signal processing



Bapatla Engineering College, Bapatla (Autonomous)

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

- **PEO-I:** Equip Graduates with a robust foundation in mathematics, science and Engineering Principles, enabling them to excel in research and higher education in Electronics and Communication Engineering and related fields.
- **PEO-II:** Impart analytic and thinking skills in students to develop initiatives and innovative ideas for Start-ups, Industry and societal requirements.
- **PEO-III:** Instill interpersonal skills, teamwork ability, communication skills, leadership, and a sense of social, ethical, and legal duties in order to promote lifelong learning and Professional growth of the students.



Bapatla Engineering College, Bapatla (Autonomous)

Department of Electronics and Communication Engineering

Project Work															
IV B. Tech - VIII Semester (Code: 20EC801/PW)															
Course Outcomes: At the end of the course, students will be able to															
CO1	Identify the real time problem related to domain knowledge and outline a solution for the problem.														
CO2	Acquire practical knowledge related to preparation of project.														
CO3	Report the outcomes of the project by means of verbal and written presentation.														
Mapping of Course Outcomes with Program Outcomes & Program Specific Outcomes															
PO's													PSO's		
CO	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO1	3	3	3	3	3	3	3	3	3	3	2	3	3	3	3
CO2	3	3	3	3	3	3	3	3	3	3	2	3	3	3	3
CO3	3	3	3	3	3	3	3	3	3	3	2	3	3	3	3
AVG	3	3	3	3	3	3	3	3	3	3	2	3	3	3	3

ACKNOWLEDGEMENTS

The successful completion of any task would be incomplete without greeting those who made it possible and whose guidance and encouragement made the effort taken a success. We express our sincere thanks to our beloved and esteemed guide **Dr. Ch. V. M. S. N. Pavan Kumar, M. Tech, Ph.D** Assistant Professor, Department of Electronics and Communication Engineering, for his excellent guidance and constant encouragement throughout the project work. His timely guidance and motivation helped us to complete the project work.

We are grateful to **Dr. B. Chandra Mohan, M. Tech, Ph.D**, Head of the Department of Electronics and Communication Engineering for his cooperation, encouragement and constant review during the project.

We are thankful to **Dr. B. Srinivas Rao, M. Tech, Ph.D**, Principal, Bapatla Engineering College, for providing excellent academic environment in the college for the successful completion of our project.

We would like to express our Sincere thanks to our project guide **Dr. Ch V. M. S. N. Pavan Kumar, M. Tech, Ph.D**, Assistant Professor, Dept. of ECE for his helpful suggestions in presenting this document. We also thank other teaching and non-teaching staff for their assistance and help extended. We thank one and all that have contributed either directly or indirectly to our project.

We extend our sincere thanks to our college management who helped to fulfill the requirements to complete the Degree.

With Regard's.....

M. Rupa Mani Latha	(Y22AEC509)
M. Pavan Krishna	(L23AEC599)
M. Kanchana	(Y22AEC514)
O. Manikanta	(Y22AEC529)

INDEX

S. NO	CONTENTS	PAGE NO
	List of Figures	i
	List of Tables	ii
	Abstract	iii
	CHAPTER 1 - INTRODUCTION	1-14
1.1	Bus Arbiter	1
1.2	Need for Arbitration in Modern Digital Systems	2
1.3	Shared Resources Access in Digital Systems	2-3
1.4	Role of Arbiter in System Performance	3-4
1.5	Resource Contention in Multi-Request Architecture	4
1.5.1	Causes of Resource Contention	4-5
1.5.2	Impact of Contention on System Throughput	5
1.6	Overview of Arbiter Architectures	5-6
1.6.1	Round Robin Arbiter	6-7
1.6.2	LRS Arbiter	7
1.6.3	Lock-Based Arbitration Mechanism	7-8
1.6.3.1	Lock Concept and Grant Retention	8
1.6.3.2	Advantages of Lock-Based Arbitration	8-9
1.6.3.3	Limitations of Lock-Based Arbitration	9
1.6.4	Enhanced Ping Lock Arbiter	9-10
1.7	Limitations of Conventional Arbitration Techniques	10-11
1.7.1	Starvation and Fairness Issues	11
1.7.2	Latency and Performance Overhead	11-12
1.7.3	Power and Area Constraints	12-13
1.7.4	Fairness Improvement Using Ping Lock	13
1.8	Motivation of Extended Ping Lock Arbiter	13-14
	CHAPTER 2 – LITERATURE SURVEY	15-25
2.1	Simple Round Robin Arbiter	15

2.1.1	Round Robin Using Priority Logic / Priority Encoder	16-17
2.1.2	Round Robin Using Without Priority Logic	17-18
2.2	Least Recent Served (LRS)	18-19
2.3	Enhanced Ping Lock Arbiter	20-21
2.4	Enhanced PLA with MUX+AND/OR Gate +Flipflops	21-23
2.5	Enhanced PLA with MUX+ Comp +AND/OR Gates	23-25
	CHAPTER 3 – ARCHITECTURE OF EPLA	26-30
3.1	Overall Architecture	26-28
3.1.1	Request Grouping and Detection Logic	28
3.1.2	Token Based Group Selection	28-29
3.1.3	Ping Pong Arbitration within Groups	29
3.1.4	Dynamic grant Lock Mechanism	30
3.1.5	Skip Logic	30
	CHAPTER 4 – IMPLEMENTATION OF Extended PLA	31-40
4.1	Tool Description	31-33
4.2	Verilog Implementation of EPLA	33-35
4.3	Working of EPLA	35-37
4.4	Methodology	37-38
4.5	RTL Schematic	39-40
4.6	FPGA Implementation	40
	CHAPTER 5 – RESULTS AND SIMULATION	41-51
5.1	Software used-Xilinx Vivado – 2020.1	41
5.2	Simulations and Analysis	41-46
5.2.1	Power Analysis	46-47
5.2.2	Timing Analysis	47-48
5.3	Hardware Verification	48-49
5.4	Comparative Analysis	49-51
	CONCLUSION	52
	FUTURE SCOPE	53

	REFERENCES	54-56
	RESUMES	57-62
	PUBLICATION DETAILS	63
	PAPER	64-76

LIST OF FIGURES

S. NO	TITLE	PAGE NO
1.1	Flow Chart of Bus Arbiter	1
1.2	Bus Arbiter	2
1.3	Block Diagram of Shared Resources	2
1.4	Flow Chart of Round-Robin Arbiter	6
2.1	Working Flow of Round-Robin Scheme	15
2.2	Simple Round Robin	15
2.3	4x4 Bus Round-Robin Arbiter	17
2.4	Architecture of LRS	20
2.5	Architecture of Enhanced PLA	21
2.6	Enhanced PLA with MUX+AND/OR Gate +Flipflops	23
2.7	Enhanced PLA with MUX+ Comparator +AND/OR Gates	24
3.1	Block Diagram of Extended PLA	26
3.2	Architecture of Extended PLA	28
3.3	Block Diagram of Ping-Pong Mechanism	29
4.1	Flow Chart of Tool Description	31
4.2	Flow of Verilog Implementation of Extended PLA	34
4.3	Schematic Diagram of Extended PLA	39
5.1	Simulation result for single requester arbitration	41
5.2	Simulation result for dual requester arbitration	42
5.3	Simulation result for multiple requester arbitration	43
5.4	Simulation result for multiple requester arbitration	44
5.5	Power report of Extended PLA	47
5.6	Timing report of Extended PLA	47
5.7	FPGA Verification of Extended PLA	48
5.8	Power distribution of Extended PLA	50
	Patent Publication details	63

LIST OF TABLES

Table. No	TITLE	PAGE NO
1	Comparative Analysis of Different Arbiters	13
2	Comparison table of Ring Counter	18
3	Comparative analysis of Existing arbiters	25
4	Arbitration Results for Different Request Combinations	46
5	Comparative Analysis	49

ABSTARCT

Efficient arbitration plays a crucial role in shared resource systems, as it determines how multiple requesters gain access to a common resource while ensuring fairness, reduced latency, and optimal power consumption. Conventional Enhanced Ping Lock Arbiters reduce arbitration overhead by holding grants for a fixed duration; however, such static allocation becomes inefficient under varying request patterns, leading to unnecessary operations and increased delays, particularly under light load conditions. This project proposes a Dynamic Group-Based Extended Ping Lock Arbiter (EPLA) for four requesters to address these limitations. In the proposed architecture, requesters are organized into groups, and a token-based mechanism is employed to select one group at a time, thereby minimizing arbitration activity. Within the selected group, a ping-pong scheduling technique ensures fair access among all requesters. Additionally, the grant duration is dynamically adjusted based on the number of active requests, improving adaptability to changing workloads. To further enhance efficiency, skip logic is incorporated to bypass inactive groups, eliminating redundant computations and reducing switching activity. A fixed priority mechanism is applied during the initial cycle to ensure deterministic system startup behaviour. The proposed design achieves a balance between fairness, performance, and power efficiency, making it well-suited for shared bus systems and Network-on-Chip (NoC) architecture.

CHAPTER-1

INTRODUCTION

1.1 Bus Arbiter

A bus arbiter is a hardware control unit responsible for managing access to a shared system bus when multiple components attempt to use it simultaneously. In a typical embedded architecture, several modules such as the processor, memory units, direct memory access (DMA) controllers, and input/output peripherals communicate through a common communication bus. The bus arbiter monitors the requests generated by these modules and determines which device is granted control of the bus at a particular moment. It receives request signals from different bus masters and issues a grant signal to one of them based on the arbitration logic implemented in the system. Once a device is granted access, it can perform data transfer operations over the bus while other devices wait until the bus becomes available again.

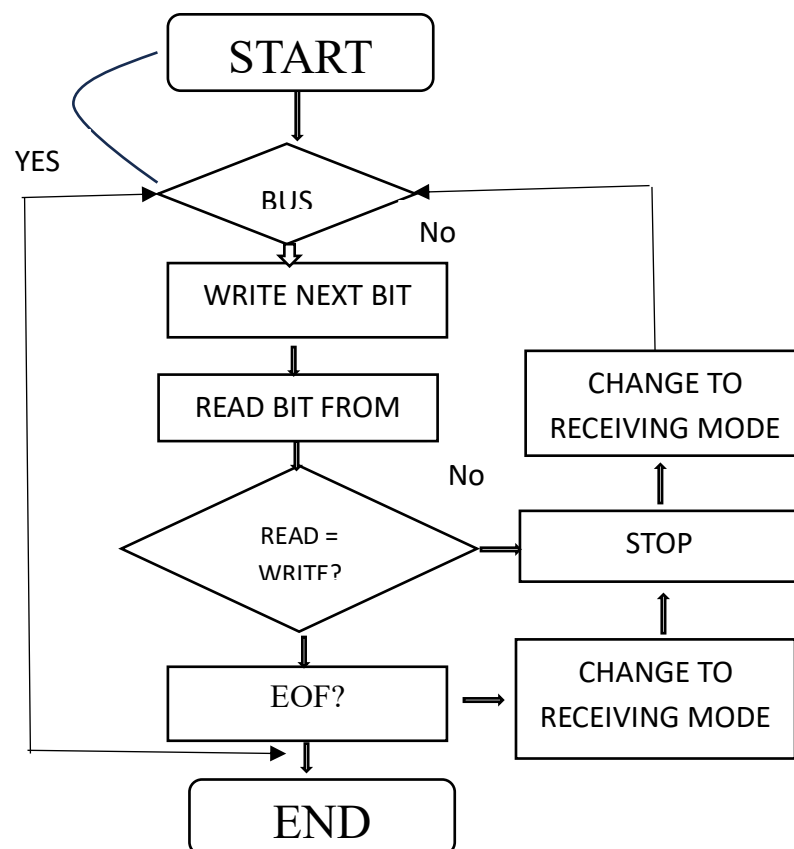


Fig 1.1 Flow chart of Bus Arbiter

1.2 Need for Arbitration in Modern Digital Systems

In modern digital systems, multiple components such as processors, memory units, peripherals, and controllers often need to share common resources like system buses, memory, or communication channels. Since only one component can use a shared resource at a time, a control mechanism is required to manage access efficiently and fairly. This mechanism is known as **Arbitration**.

As digital systems have become more complex and faster, the number of devices requesting access to shared resources has increased significantly. Without arbitration, simultaneous access requests could lead to data corruption, signal conflicts, unpredictable behaviour, or complete system failure. Arbitration ensures that access is granted to one requester at a time based on predefined rules.

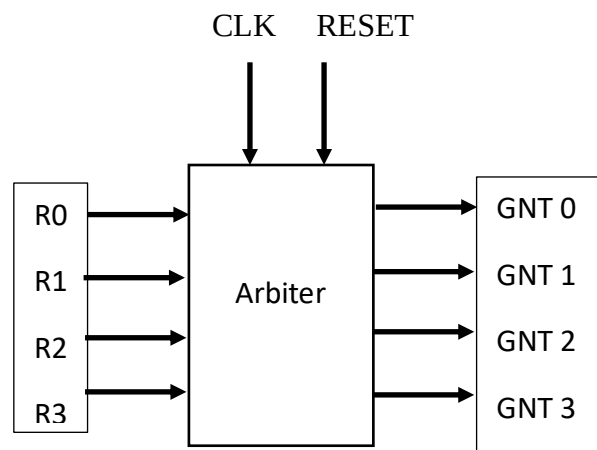


Fig 1.2 Bus Arbiter

1.3 Shared Resource Access in Digital Systems

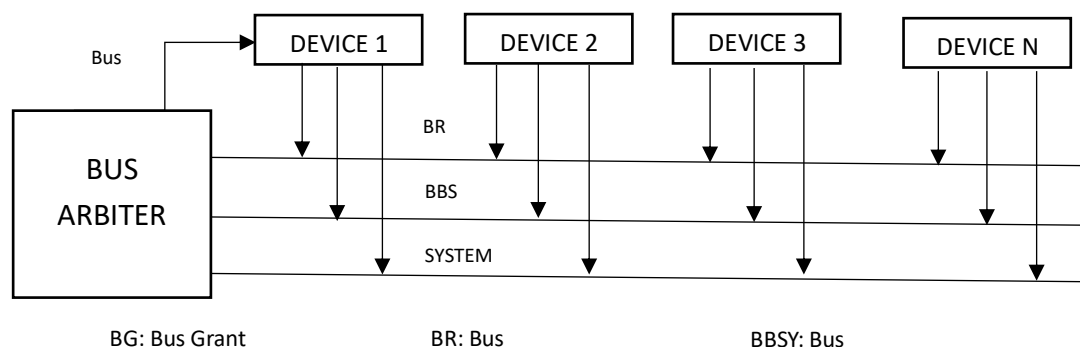


Fig 1.3 Block Diagram of Shared Resources

In digital systems, a **shared resource** is any hardware or communication element that is used by more than one functional unit. Common shared resources include system buses, memory modules, input/output (I/O) devices, caches, and communication channels. Since these resources cannot serve multiple requests simultaneously in most designs, controlled access is essential.

Multiple components such as processors, direct memory access (DMA) controllers, and peripheral devices may request access to a shared resource at the same time. If these requests are not properly managed, it can lead to bus contention, data collisions, incorrect data transfers, and unpredictable system behaviour. Therefore, a well-defined access control mechanism is required. Shared resource access is typically managed using arbitration logic, which monitors incoming requests and grants access to one requester at a time according to predefined policies such as priority, fairness, or time-based allocation. Once access is granted, other requesting units are temporarily blocked until the resource becomes free again.

1.4 Role of Arbiter in System Performance

The **Arbiter** plays a critical role in determining the overall performance of a digital system by controlling how shared resources are accessed. It acts as a decision-making unit that evaluates multiple access requests and grants permission to one requester at a time based on a defined arbitration scheme.

An efficient arbiter directly improves **system throughput** by minimizing access delays and reducing idle time of shared resources. By quickly resolving contention among multiple requesters, the arbiter ensures that resources such as buses and memory are utilized effectively. Poorly designed arbitration, on the other hand, can introduce unnecessary latency and become a performance bottleneck.

The arbiter influences system **response time** and **fairness** by controlling access to shared resources. Using priority-based, round-robin, or dynamic arbitration techniques, it ensures timely access for critical components while preventing starvation of lower-priority devices. This balance is essential in real-time and high-speed systems where predictable performance is required. Additionally, a well-designed arbiter supports

scalability by handling increased request traffic as more devices are added, while preventing access conflicts and maintaining reliable system operation.

1.5 Resource Contention in Multi-Request Architecture

In multi-request architectures, multiple functional units such as processors, memory controllers, DMA units, and peripheral devices may simultaneously request access to shared system resources. Since these resources are limited and can typically be used by only one requester at a time, resource contention becomes a major challenge. It occurs when two or more requesters compete for the same resource, leading to increased latency, reduced throughput, and unpredictable system behaviour if not properly managed. As the number of requesters increases, the complexity of contention also rises. These architectures are common in multicore processors, System-on-Chip (SoC) designs, and high-performance embedded systems, where resources like buses, memory, caches, and interconnects are shared. To manage contention, arbitration mechanisms are used to grant access based on predefined policies such as fixed priority, round-robin, or dynamic priority, ensuring orderly access, data integrity, and balanced system performance.

Effective management of resource contention provides the following advantages:

- Reduces access delays and bottlenecks
- Improves overall system throughput
- Ensures fairness among multiple requesters
- Maintains reliable and predictable system operation

1.5.1 Causes of Resource Contention

Resource contention in digital systems occurs when multiple components try to access the same shared resource simultaneously. One major cause is the limited availability of shared resources such as system buses, memory, and I/O interfaces, which can typically serve only one requester at a time. Another key factor is the increasing number of requesters in modern systems like multicore processors and SoC designs, which raises the likelihood of simultaneous access requests. High-speed and parallel operation of components further intensifies contention, as requests are generated more frequently.

Additionally, uneven access patterns and priority differences can cause some components to dominate the resource, increasing waiting time for others.

1.5.2 Impact of Contention on System Throughput

- **Single Access Restriction**

Only one requester can access a shared resource at a time, forcing others to wait.

- **Latency Increase**

Contention adds waiting time, raising overall system latency.

- **Idle Processing Units**

While waiting for resource access, processing units remain idle, reducing efficiency.

- **Throughput Reduction**

Useful work completed per unit time decreases due to delays.

- **Resource Bottlenecks**

Heavily used resources (buses/memory) become bottlenecks, limiting performance even with more processors.

- **Unfairness and Starvation**

Without proper arbitration, some requesters may be denied access repeatedly.

- **Arbitration Overhead**

Continuous conflict resolution consumes cycles, reducing time available for actual data processing.

1.6 Overview of Arbiter Architectures

An arbiter is a control mechanism used in digital systems to manage access to a shared resource when multiple requesters attempt to use it simultaneously. Arbiter architectures define the method by which the arbiter evaluates requests and grants access, ensuring proper coordination, efficiency, and reliability of the system. Different architectures are designed to meet specific requirements such as speed, fairness, simplicity, and scalability. In a fixed priority arbiter, each requester is assigned a permanent priority, and the arbiter always grants access to the requester with the highest priority. This architecture is simple and fast but may cause starvation for lower-priority

requesters. Round robin arbiters overcome this limitation by rotating the priority among all requesters, ensuring fair access to the shared resource. After each grant, the next requester in sequence is given priority, making this architecture suitable for systems requiring fairness.

Dynamic priority arbiters adjust priorities based on system conditions such as waiting time or frequency of requests. This approach improves system performance and reduces starvation but increases design complexity. Daisy chain arbiters connect requesters in a serial manner, where priority depends on the physical position in the chain. Although simple to implement, this architecture suffers from increased delay as the number of requesters grows.

1.6.1 Round Robin Arbiter

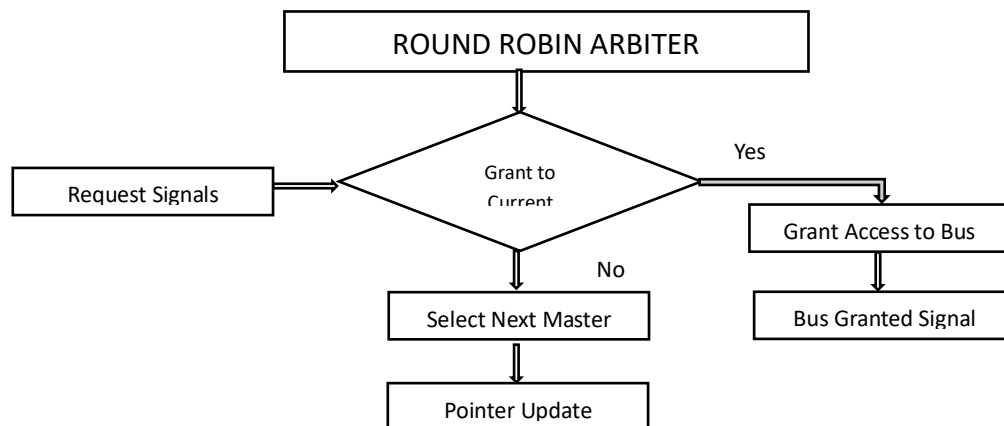


Fig 1.4 Flow chart of Round Robin Arbiter

A Round Robin Arbiter is an arbitration technique used in digital systems to provide fair access to a shared resource among multiple requesters. Unlike fixed priority arbiters, it rotates priority in a cyclic order, ensuring that each requester gets an equal chance of access. In this method, requesters are arranged in a sequence, and when multiple requests occur, the arbiter grants access starting from the next requester after the last granted one. After each grant, the priority pointer moves forward, preventing starvation and maintaining fairness. Although it requires additional logic such as a pointer or counter, this slightly increased complexity is justified by improved resource utilization and system reliability. The arbitration is typically completed within a single

clock cycle, making it suitable for synchronous systems. Round robin arbiters are widely used in shared buses, multi-core processors, memory controllers, and communication systems where fair resource allocation is important. Overall, it provides a good balance between fairness, performance, and implementation complexity.

1.6.2 LRS Arbiter

An LRS (Least Recently Served) Arbiter is an arbitration technique designed to improve fairness by granting access to the requester that has not been served for the longest time. Unlike fixed priority arbiters, which may cause starvation, and round robin arbiters, which follow a strict cyclic order, the LRS arbiter dynamically selects the requester based on service history. In an LRS arbiter, the system maintains information about when each requester was last granted access. When multiple requests are active, the arbiter compares their service history and grants the shared resource to the requester that was least recently served. After a grant is issued, the service record for that requester is updated, making it the most recently served.

This approach ensures a high level of fairness and effectively eliminates starvation, especially in systems with irregular or bursty request patterns. However, implementing an LRS arbiter requires additional storage and comparison logic to track and evaluate service history, which increases hardware complexity compared to round robin or fixed priority arbiters. LRS arbiters are commonly used in high-performance processors, memory controllers, and interconnection networks where fairness and balanced resource utilization are critical. Overall, the LRS arbiter provides adaptive and fair arbitration at the cost of increased implementation complexity, making it suitable for advanced digital systems.

1.6.3 Lock-Based Arbitration Mechanism

A lock-based arbitration mechanism is used in digital systems to manage access to shared resources by allowing a requester to retain control for a defined duration. Unlike conventional arbitration, which evaluates requests for every transaction, it uses a lock signal to prevent re-arbitration during critical or multi-cycle operations. Once granted access, a requester can hold the resource until its operation is complete, while other requests are temporarily blocked. This ensures uninterrupted execution of atomic or burst transactions and maintains data consistency. Lock-based arbitration reduces

overhead by minimizing frequent grant switching, thereby improving system performance, especially in high-speed buses and memory systems. However, if not carefully designed, prolonged locking can increase latency and reduce fairness, so it is often combined with fairness mechanisms.

1.6.3.1 Lock Concept and Grant Retention

The lock concept in arbitration is a technique that allows a requester to retain exclusive access to a shared resource for a specific period, ensuring uninterrupted execution of critical or multi-cycle operations. In this

approach, once a requester is granted access, it can assert a lock signal, which tells the arbiter to hold the grant on that requester until the operation is complete.

Grant retention means that the arbiter does not re-evaluate other requests during the lock period. Even if other requesters issue requests, they must wait until the current lock is released. This ensures data consistency, avoids conflicts, and allows operations such as burst transfers, atomic transactions, or memory updates to execute without interruption.

The lock duration can vary depending on the type of operation:

- Short locks for small or single-cycle operations.
- Extended locks for multi-cycle bursts or critical sequences.

While grant retention improves throughput and reduces arbitration overhead, it can also introduce fairness issues if locks are held too long, delaying other requesters. To address this, modern arbiters often combine lock-based grants with fairness mechanisms like round robin or least-recently-served logic.

1.6.3.2 Advantages of Lock-Based Arbitration

Exclusive Access Retention – The requester can hold the shared resource for a specific duration.

Uninterrupted Execution – Critical or multi-cycle operations can complete without interruption.

Data Consistency – Maintains correct data during burst transfers, atomic operations, and memory updates.

Reduced Arbitration Overhead – Minimizes repeated arbitration processes, improving system efficiency.

Predictable Timing – Provides deterministic access, which is important for real-time embedded systems.

Support for Burst Transactions – Enables continuous multi-cycle data transfers for better performance.

1.6.3.3 Limitations of Lock-Based Arbitration

Increased Latency – Other requesters may experience delays because the resource remains locked until the current operation finishes.

Fairness Issues – Some requesters may repeatedly wait if the lock is held frequently or for long durations.

Hardware Complexity – Additional control logic is required to manage lock signals and arbitration control.

Risk of Deadlock or Starvation – Improper design can cause situations where some requesters never get access to the resource.

Reduced Resource Utilization – Holding the lock for long periods may prevent efficient use of the shared resource, especially when traffic is low.

1.6.4 Enhanced Ping Lock Arbiter

The Ping Lock Arbiter is an advanced arbitration technique used to efficiently manage access to shared resources while supporting multi-cycle or burst transactions. It combines the features of lock-based arbitration with a ping mechanism to monitor pending requests and maintain fairness among multiple requesters. In this approach, when a requester is granted access to a shared resource, it can assert a lock signal to retain the grant for the duration of a critical or multi-cycle operation. During this locked period, the arbiter continuously monitors or “pings” other requesters to detect their requests. Once the lock is released, the arbiter quickly allocates the resource to the next

requester based on priority or fairness rules, ensuring that no requests are lost and that the system operates efficiently.

The ping mechanism in bus arbitration is used to manage communication among multiple devices connected to a shared bus. Before transmitting data, a device sends a small ping signal to check whether the bus is free and to detect other active devices on the network. Other nodes respond if they are currently active or if the bus is occupied. Based on these responses, the arbitration logic determines which device is allowed to access the bus, typically using predefined priority or arbitration rules when multiple devices request access simultaneously. This mechanism helps maintain orderly communication, prevents data collisions, and ensures efficient operation in multi-device systems by acting as a pre-transmission check.

1.7 Limitations of Conventional Arbitration Techniques

Conventional arbitration techniques are widely used in digital systems to control access to shared resources such as buses and memory. Although these techniques are simple and effective for basic system designs, they exhibit several limitations when applied to modern high-speed and high-complexity systems. One major limitation is starvation, which commonly occurs in fixed priority arbitration. In such schemes, lower-priority requesters may never receive access if higher-priority requests are continuously present. This leads to unfair resource utilization and degraded system performance. Even though round robin arbitration improves fairness, it may still introduce delays when a large number of requesters are present.

Another limitation is increased latency. In techniques like daisy chain arbitration, the grant signal must pass through multiple devices, causing propagation delays that grow with system size. This makes conventional arbiters unsuitable for systems with a high number of requesters or strict timing requirements. Limited scalability is also a concern. As the number of requesters increases, the hardware complexity and arbitration time increase significantly in conventional methods such as matrix and dynamic priority arbiters. This limits their practical use in large-scale or highly parallel systems. Conventional arbitration techniques also suffer from inefficient handling of burst and multi-cycle transactions. Frequent re-arbitration between transfers increases overhead and reduces throughput. Additionally, most

traditional arbiters lack support for atomic or locked transactions, which are essential in modern processors and SoC designs. Finally, many conventional arbiters are not adaptive to changing system conditions. They do not consider factors such as request history, workload variations, or quality-of-service requirements. As a result, they may fail to provide optimal performance in dynamic and real-time environments.

1.7.1 Starvation and Fairness Issues

Starvation and fairness concerns represent critical drawbacks in conventional arbitration techniques for shared resource systems. Starvation occurs when a requester is repeatedly denied access because higher-priority requesters dominate arbitration. This phenomenon is particularly evident in fixed-priority arbiters, where low-priority requesters may be perpetually blocked under heavy system load. Such behavior leads to uneven resource utilization and can significantly degrade overall system performance.

A lack of fairness manifests as increased latency and unpredictable response times, which are undesirable in real-time and high-performance systems where balanced and timely access is essential. While round-robin arbitration improves fairness by rotating priorities, it may still suffer delays when the number of requesters is large or when certain requesters generate continuous traffic. Similarly, daisy-chain arbiters inherently favour requesters positioned closer to the arbiter, resulting in biased access patterns.

These starvation and fairness issues contribute to system inefficiencies, reduced throughput, and failure to meet timing constraints.

1.7.2 Latency and Performance Overhead

Latency and performance overhead are significant limitations of conventional arbitration techniques in shared resource systems. Latency refers to the time a requester must wait before gaining access to a shared resource, while performance overhead includes the extra time and hardware effort required to perform arbitration. In many traditional arbitration schemes, such as daisy chain and fixed priority arbiters, the arbitration decision process can introduce additional delays, especially when multiple requesters are active. In daisy chain arbitration, the grant signal propagates sequentially

through each requester, increasing delay as the number of requesters grows. This results in higher access latency and slower system response.

Frequent re-arbitration is another major source of performance overhead. In conventional techniques, arbitration is often performed for every transaction or data transfer. This repetitive decision-making consumes clock cycles and reduces overall system throughput, particularly in systems that support burst or multi-cycle transactions. Additionally, some arbitration methods require complex comparison and control logic, which increases critical path delay and power consumption. As system speed and integration scale increase, these delays become more pronounced and can limit maximum operating frequency.

1.7.3 Power and Area Constraints

Power and area constraints are important limitations of conventional arbitration techniques, especially in modern integrated circuits and SoC designs. As the number of requesters increases, the arbitration logic becomes more complex, leading to higher silicon area usage and increased power consumption. Conventional arbiters such as matrix arbiters and dynamic priority arbiters require multiple comparators, registers, and control logic to evaluate requests and priorities. This additional hardware occupies more chip area and contributes to higher static and dynamic power dissipation. In power-sensitive applications, such as mobile and embedded systems, this becomes a critical drawback.

Frequent arbitration activity also increases switching activity in the control logic, resulting in higher dynamic power consumption. In systems where arbitration is performed every clock cycle, the continuous toggling of signals leads to unnecessary energy loss and reduced battery life. Furthermore, larger arbiter designs increase routing complexity, which adds to interconnect power and can negatively impact timing. As a result, conventional arbitration techniques struggle to meet strict power and area budgets while scaling to support a large number of requesters.

Table 1: Comparative Analysis of Different Arbiters

Metric	Round Robin	LRS (Least Recently Served)	Enhanced Ping Lock Arbiter
Switching Activity	Moderate	High	Low
Complexity	Moderate	High	Moderate
Fairness	High	Medium	High
Starvation	Low	Very Low	Low
Scalability	Good	Very Good	Good
Latency	Medium	Variable	Low
Power Constraint	Moderate	High	Low
Area Constraint	Moderate	High	Moderate

1.7.4 Fairness Improvement Using Ping Lock

The ping lock mechanism enhances fairness in bus arbitration systems where multiple devices compete for access to a shared bus. In such environments, high-priority devices may dominate the bus, causing lower-priority devices to experience delays. Ping lock addresses this issue by allowing a device that wins arbitration to temporarily lock the bus until its data transfer is completed. This prevents repeated interruptions by higher-priority devices and ensures uninterrupted transmission. As a result, all devices receive a fair opportunity to access the bus, which helps balance bus utilization, reduces starvation of lower-priority devices, and improves overall system efficiency while maintaining smooth and reliable data communication.

1.8 Motivation for Extended Ping Lock Arbiter

Traditional bus arbitration techniques such as fixed priority, round-robin, and basic ping-based arbiters are widely used to manage access to shared resources. However, in

modern systems with multiple high-speed devices requesting the bus simultaneously, these methods may lead to increased latency, unfair access, and resource contention. Higher-priority devices can dominate the bus, causing delays or starvation for lower-priority requesters. The Extended Ping Lock Arbiter (EPLA) is motivated by the need to improve fairness, reduce latency, and enhance overall system efficiency. By combining a lock mechanism with an enhanced ping-based monitoring approach, it allows a requester to hold the resource during critical operations while still tracking pending requests, enabling faster and more balanced allocation of the bus once the lock is released.

CHAPTER 2

LITERATURE SURVEY

2.1 Simple Round Robin Arbiter

Round Robin (RR) arbitration scheme. The round robin technique ensures fairness among multiple masters by allocating bus access in a cyclic order. In this method, each master is given an equal opportunity to access the bus. Unlike fixed priority arbitration, where certain masters always have higher priority, the round robin algorithm rotates the priority among masters after every successful bus transaction. [1][2]

The main advantage of the round robin scheme is that it prevents starvation, which occurs when a low-priority master never receives access to the shared resource. By rotating priority sequentially, the round robin arbiter guarantees that every requesting master will eventually receive bus access within a limited number of arbitration cycles.

For example, consider a system with four masters M0, M1, M2, and M3 connected to a shared bus. The arbiter grants bus access in the following cyclic sequence:

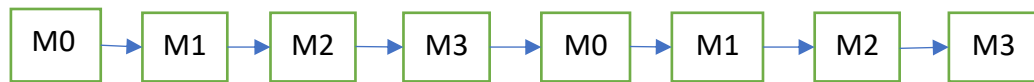


Fig 2.1 Working flow of Round Robin Scheme

If a master does not request the bus during its turn, the arbiter skips that master and grants the bus to the next requesting master in the sequence. Because of its fairness and simple design, the round robin arbiter is widely used in **multiprocessor systems, network**

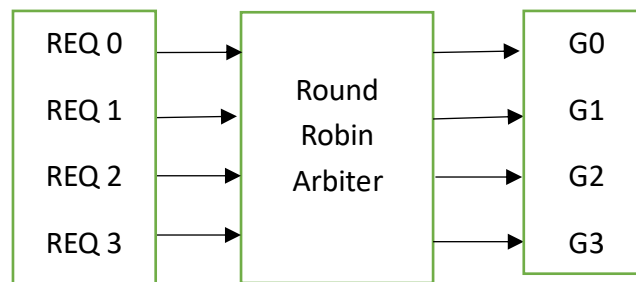


Fig 2.2 Simple Round Robin

2.1.1 Round Robin using Priority Logic / Priority Encoder

The priority of inputs are placed in descending order from in[0] to in[3] in the priority logic blocks (Priority Logic 0 through 3) shown in Figure 2.3. Thus, in[0] has the highest priority, in[1] has the next priority.

To implement a BA, we employ the token concept from a token ring in a network. The possession of the token allows a priority logic block to be enabled. Since each priority logic block has a different order of inputs (request signals), the priority of request signals varies with the chosen priority logic block. The token is implemented in a 4-bit ring counter as shown in Figure 2.3. The outputs (four bits) of the ring counter act as the enable signals to the priority logic blocks. Thus, only one enabled priority logic block can assert a grant signal. The ack signal to the bus arbiter is delayed by one arbitration cycle by a D flip-flop as shown in Figure 2.3. The delayed ack signal pulls a trigger to the ring counter so that the content of the ring counter is rotated one bit. Thus, the token bit is rotated left each cycle, with 4'b1000 rotating to 4'b0001 in Figure 2.3 and the token is initialized to one at the reset phase (e.g., 4'b0001 for four-bit ring counter) so that there is only one '1' output by the ring counter. In the round-robin algorithm, each master must wait no longer than (M1) time slots, the period of time allocated to the chosen master, until the next time it receives the token (i.e., highest priority). The assigned time slot can also be yielded to another master if the owner of the time slot has nothing to send. This protocol guarantees a dynamic priority assignment to bus masters (requestors) without starvation. [1][2]

The main disadvantage of a Round Robin arbiter using a priority encoder is increased hardware complexity and delay due to the combinational logic required to determine the highest priority request.

- The priority encoder must **scan all request inputs** to determine the highest priority.
- This increases the **propagation delay**, especially when many request lines are present.

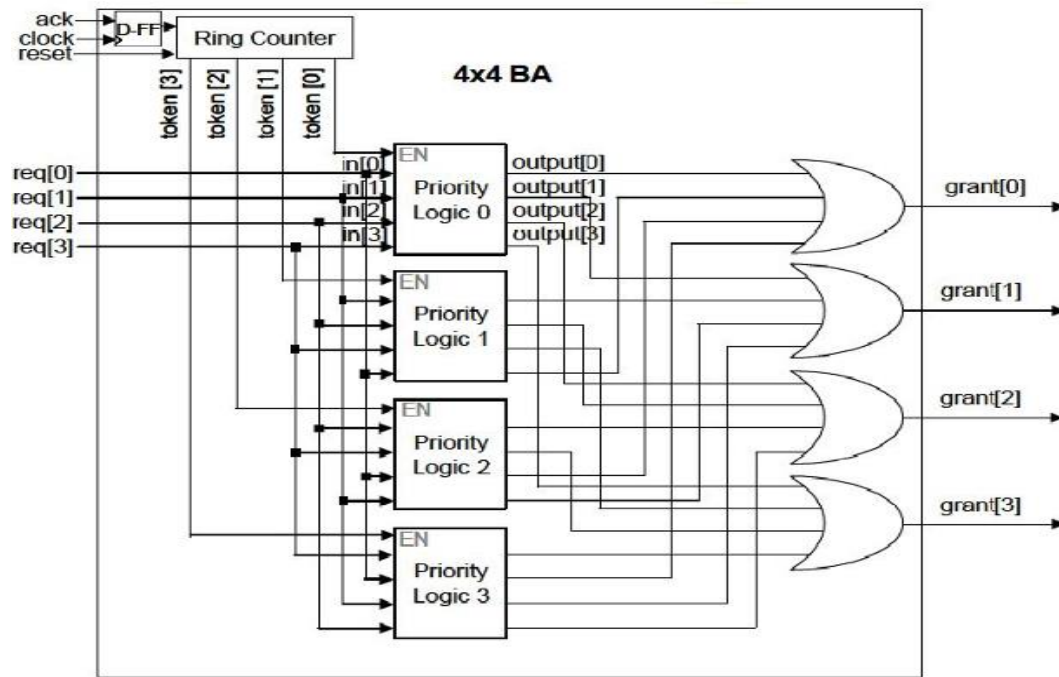


Fig 2.3 4X4 Bus Round Robin Arbiter

2.1.2 Round Robin using Without Priority Encoder

Another implementation of the round robin arbiter eliminates the use of a priority encoder and instead uses **sequential logic circuits such as ring counters, flip-flops, and logic gates**. In this design, a **token passing mechanism** is used to determine the master that has the highest priority during a particular arbitration cycle.

According to the work by **Shashidhar et al., [1][2]** the round robin arbiter can be implemented using a **ring counter, D flip-flops, priority logic blocks, and OR gates**. The ring counter generates a rotating token that determines which master has the highest priority at a given time. The outputs of the ring counter enable one priority logic block at a time, allowing that master to generate the grant signal. **[1][2]**

After each arbitration cycle, the ring counter shifts its output to the next position, thereby passing the token to the next master in the sequence. This mechanism ensures fair access to the shared bus while maintaining simple hardware implementation.

Example

Consider a system with four masters **M1, M2, M3, and M4**. A 4-bit ring counter produces the following sequence:

0001 → 0010 → 0100 → 1000 → 0001

Each state indicates the master that currently has the highest priority.

Table2: Comparison table of Ring Counter

Ring Counter Output	Request with Highest Priority
0001	Req1
0010	Req 2
0100	Req 3
1000	Req 4

Assume the request signals are:

Req 0=0

Req 1=1

Req 2=1

Req 3=0

If the ring counter output is **0001**, master Req 0 has priority but does not request the bus. The token then moves to **0010**, giving priority to **REQ 2**, which receives the grant signal. After M2 completes its transaction, the ring counter shifts to **0100**, giving priority to **Req 3**.

2.2 Least Recent Served (LRS)

The **Least Recently Served (LRS)** arbitration scheme is a dynamic priority technique that grants access to the master that has not been served for the longest time. Unlike fixed priority arbitration, the LRS scheme continuously updates the priority of masters based on their service history. In this approach, the arbiter maintains a record of previously served masters and selects the master with the longest waiting time when multiple requests occur simultaneously. This mechanism ensures fair allocation of bus resources and eliminates starvation problems in shared communication systems.

According to the research on arbiter architectures for Network-on-Chip routers, fairness among competing requests is a major requirement for high-performance communication systems. [3]

In LRS arbitration, the arbiter continuously monitors the request signals from all masters. When a request arrives, the arbiter compares the service history of each master and determines which master has waited the longest since its last service. The arbiter then grants access to that master while updating the service history table. This dynamic updating mechanism ensures that each master receives fair access to the shared bus over time. [3]

For example, consider a system with four Requests from four masters **Req 0, Req 2, Req 3, and Req 4** connected to a shared bus. Assume that the previous order of bus access is:

Req 2 → Req 0 → Req 3 → Req 1

In this scenario, **Req 2** was served most recently while **Req 1** was served earlier than others. Therefore, when a new arbitration cycle begins and multiple requests are present, the arbiter grants the bus to the master that was served least recently. After the selected master completes its transaction, the service history is updated accordingly.

The LRS arbitration technique is particularly useful in **multiprocessor systems, on-chip communication networks, and router architectures**, where multiple requests compete for limited resources. By dynamically updating priority levels, the LRS scheme ensures balanced access among masters and improves overall system fairness. However, the LRS arbitration scheme also has certain limitations. Since the arbiter must maintain and update the service history of each master, the hardware implementation may require additional storage and control logic. This increases design complexity compared to simpler arbitration schemes such as fixed priority arbitration. Furthermore, in systems with a large number of masters, maintaining the service history may increase arbitration delay. [3]

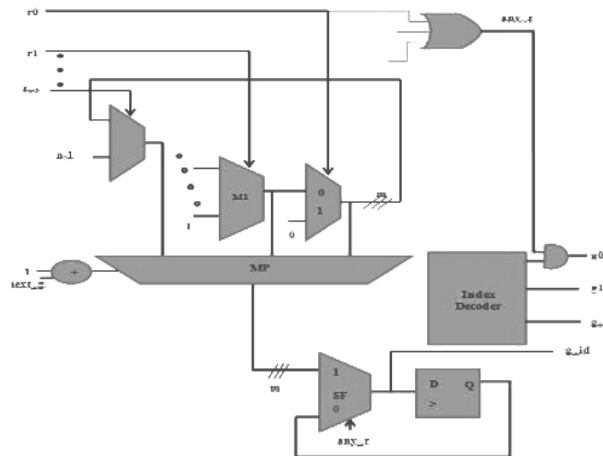


Fig 2.4 Architecture of LRS

2.3 Enhanced Ping Lock Arbiter

The design of the enhanced ping lock arbiter combines multiple **two-bit arbiters (PLA₂ units)** to construct higher-bit arbitration structures. Each two-bit arbiter generates multiple outputs including the **grant signal (g)**, **enable output (eo)**, **arbitration grant signal (AG)**, and **lock output (lo)**. Among these signals, the lock output plays a crucial role in controlling the priority update process and maintaining fairness during arbitration. When the lock signal is asserted, the priority register value is preserved in the next clock cycle, ensuring that the correct requester receives the grant signal during the arbitration cycle. [5][6]

The architecture of the enhanced ping lock arbiter includes both **PLA₂ leaf blocks** and **PLA₂ intermediate blocks**, which together form a hierarchical arbitration structure. In this design, request signals from multiple input channels are processed sequentially, and the arbiter generates corresponding grant signals in each clock cycle. For example, when four request inputs are active simultaneously, the arbiter generates grant outputs sequentially such as **g = 1000, 0001, 0010, and 0100** in consecutive clock cycles, thereby ensuring fair distribution of bus access among all requesters. Performance evaluation of different arbitration techniques shows that the enhanced ping lock arbiter provides **better fairness and balanced performance compared to conventional arbiters**. Experimental results obtained through Verilog simulation and FPGA synthesis indicate that the EPLA achieves **high speed, reduced execution delay, and efficient resource utilization**. Compared to matrix and distributive round robin

arbiters, the enhanced ping lock arbiter offers improved fairness and relatively compact chip size while maintaining high operating frequency.

Although the enhanced ping lock arbiter improves fairness and system performance, it also introduces additional control logic for lock signal management and priority updating. This slightly increases design

complexity compared to simpler arbitration mechanisms. Nevertheless, due to its balanced performance and fairness properties, the enhanced ping lock arbiter is considered a suitable arbitration technique for **high-speed SoC communication systems, network routers, and shared resource scheduling architectures**.

However, the main Disadvantage with this is, When the number of requesters increases, the **hierarchical PLA₂ structure becomes more complex**.

Scaling the design may increase **routing complexity and verification effort**.

Although the Enhanced Ping Lock Arbiter improves fairness and performance, it introduces additional hardware complexity and slightly higher delay due to the implementation of lock control logic and priority update mechanisms. [5][6]

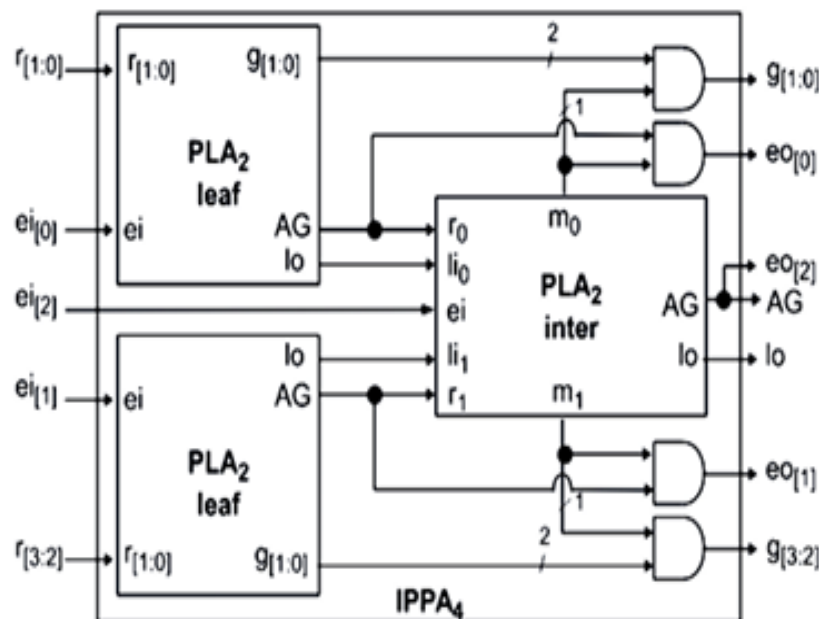


Fig 2.5 Architecture of Enhanced PLA

2.4 EPLA With MUX + AND/OR Gates + Flip Flops

The proposed structural EPLA arbiter is designed using basic digital building blocks such as D flip-flops, combinational logic, and a small counter. This structural approach

makes the design suitable for hardware implementation and synthesis in VLSI systems. The arbiter accepts four request signals and produces a one-hot encoded grant output, ensuring that only one requester is served at a time. The design incorporates token-based group selection, intra-group arbitration, and a lock mechanism to improve overall efficiency.

One of the key features of this arbiter is the use of a first-cycle priority mechanism. Immediately after reset, the arbiter operates in a fixed priority mode, where requests are served based on a predefined order. This ensures deterministic behaviour during initialization and avoids uncertainty in grant allocation. Once the first request is served, the arbiter transitions into its normal operating mode, which is governed by token-based arbitration and fairness mechanisms. [6]

The arbiter divides the requesters into two groups and uses a token signal to alternate priority between these groups. This grouping strategy reduces complexity while maintaining fairness. If both groups have active requests, the token determines which group gets access. However, if only one group has active requests, the arbiter employs a single-request bypass mechanism, allowing immediate access without waiting for token switching. This significantly reduces latency and improves responsiveness.

Within each group, arbitration is handled using pointer-based logic. The pointer toggles between requesters after each grant, ensuring that all requesters within a group receive fair access over time. This eliminates starvation and provides a balanced distribution of resources. The combination of token-based inter-group arbitration and pointer-based intra-group arbitration results in an efficient and fair arbitration scheme.

Another important feature of the design is the locking mechanism implemented using a 2-bit counter. Once a request is granted, the arbiter enters a locked state, during which the grant is held for a fixed number of clock cycles. This is particularly useful for burst transactions, where a master needs continuous access to the bus. The counter decrements on each clock cycle and releases the lock when it reaches zero, allowing the arbiter to serve new requests.

The grant output is generated using combinational logic in the unlocked state and is latched during the locked state. This ensures stable output during lock periods while maintaining fast response during normal operation. The one-hot encoding of the grant

signal simplifies downstream decoding and reduces the possibility of contention on the shared resource.

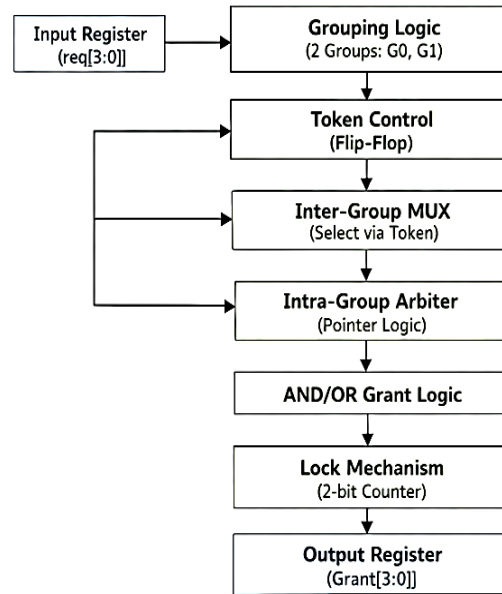


Fig 2.6 Enhanced PLA With MUX + AND/OR + Flip Flops

The structural EPLA arbiter offers several advantages over traditional arbitration schemes. It provides improved fairness through token and pointer mechanisms, reduced latency through single-request bypass, and stable operation through locking. However, these benefits come at the cost of increased hardware complexity and slightly higher area due to additional flip-flops and counters. Despite these drawbacks, the design is well-suited for modern SoC applications where performance and efficiency are critical.

2.5 Enhanced PLA With MUX + Comparator + AND/OR

The presented design implements a behavioural EPLA arbiter using Verilog HDL. It accepts four request inputs and produces a one-hot encoded grant output. The design incorporates grouping of request signals, token-based arbitration, pointer-based fairness, and a lock mechanism. These features collectively improve performance and ensure balanced resource allocation among competing masters.

The arbiter divides the four request inputs into two groups: Group 0 consisting of req[0] and req[1], and Group 1 consisting of req[2] and req[3]. A token signal is used to determine which group gets priority during arbitration. When both groups have active

requests, the token selects the group to be served. If only one group is active, the arbiter skips token dependency and directly grants access, improving latency. [6]

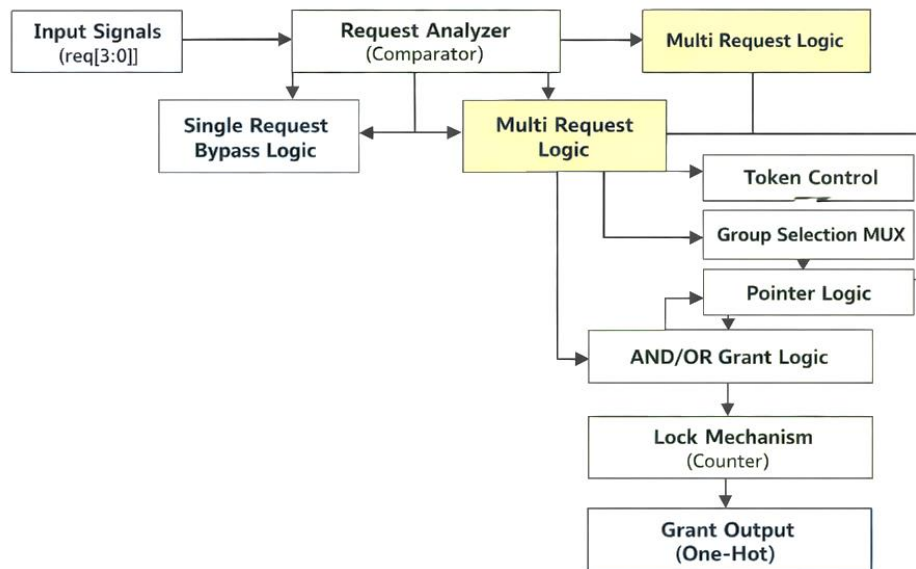


Fig 2.7 Enhanced PLA With MUX + Comp + AND/OR Gates

A key optimization in this design is the single request bypass mechanism. When only one request is active among all inputs, the arbiter directly grants access without enabling the lock mechanism. This reduces

unnecessary waiting cycles and enhances system responsiveness. The grant is generated using a simple case statement, ensuring fast decision-making in such scenarios.

For multiple simultaneous requests, the arbiter uses a lock mechanism controlled by a counter. Once a request is granted, the arbiter enters a locked state, during which the same request is held for a fixed number of clock cycles. This is particularly useful for burst transactions, where continuous access is required. The lock counter decrements each cycle and releases the lock once it reaches zero.

Within each group, fairness is maintained using pointer-based arbitration. The pointer toggles after each completed grant, ensuring that requests within the same group are served alternately. This prevents starvation and ensures equal opportunity for all requesters within a group. The pointer update occurs after the lock period ends, maintaining stability during active transactions.

The token mechanism ensures fairness between groups. After completing a transaction, the token switches to the other group if it has pending requests. If the selected group has no active requests, the token is automatically redirected to the active group. This dynamic adjustment avoids idle cycles and improves overall throughput. [6]

The grant output is maintained as a one-hot encoded signal, ensuring that only one master is granted access at any given time. During the lock period, the grant remains stable, preventing fluctuations and ensuring reliable operation. The design uses sequential logic to maintain state variables such as token, pointers, and lock status, making it suitable for synchronous digital systems.

Table 3: Comparative analysis of Existing arbiters

Techniques	Dynamic power	Static power	Data Path Delay	No. of LUTS And SLICES
REFERENCE 1 Technique 1 Simple round robin scheme	0.963w	0.084w	5.564ns	Luts=4 Slices=3
Technique 2 Round Robin Scheme (with priority)	0.941w	0.083w	5.731ns	Luts=4 Slices=3
Technique 3 Without priority encoder	0.963w	0.083w	5.564ns	Luts=5 Slices=3
REFERENCE 2 LRS Priority Scheme	5.516w	0.083w	4.104ns	Luts=13 Slices=7
REFERENCE 3 Technique 1 4-bit Extended PLA With MUX+COMP+AND/OR GATES	1.178w	0.083w	4.857ns	Luts=8 Slices=6
Technique 2 4-bit Extended PLA With MUX++AND/OR GATES	0.633w	0.083w	4.129ns	Luts=13 Slices=6

CHAPTER-3

ARCHITECTURE OF EPLA

3.1 Overall Architecture

The overall architecture of the proposed **Extended Ping Lock Arbiter (EPLA)** is designed to efficiently manage access to a shared resource when multiple requesters attempt to use it simultaneously. In modern digital systems such as System-on-Chip (SoC), Network-on-Chip (NoC), and FPGA based designs, several modules may generate requests at the same time to access a common resource like a bus, memory, or communication channel. Since only one requester can use the resource at a time, an arbiter is required to decide which requester should be granted access.

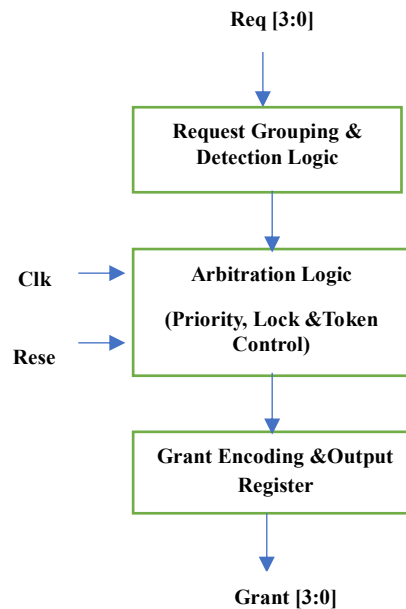


Fig 3.1 Block Diagram Of Extended PLA

The proposed EPLA architecture introduces a group-based arbitration mechanism combined with token-based selection and dynamic grant lock control. This approach helps to reduce unnecessary arbitration operations and switching activity while maintaining fairness among requesters.

In this architecture, the four requesters are divided into two groups to simplify the arbitration process. The arbiter first selects one of the groups using a token-based

mechanism, and then a requester within the selected group is chosen using a ping-pong pointer technique. This two-level arbitration strategy reduces the complexity compared to evaluating all requesters simultaneously.

Another important feature of the proposed architecture is the dynamic grant lock mechanism. Instead of keeping the grant for a fixed number of clock cycles, the lock duration is dynamically adjusted depending on the number of active requests. When the number of requests is high, the grant is held for a longer duration to reduce repeated arbitration. When fewer requests are present, the lock duration is reduced to improve system responsiveness.

The architecture also includes skip logic, which allows the arbiter to skip inactive groups. If a group does not contain any active requests, the token automatically switches to the other group. This helps to eliminate unnecessary arbitration cycles and further reduces switching activity and power consumption.

The complete architecture consists of three main functional blocks:

- **Request Grouping and Detection Logic**
- **Arbitration Logic with Token and Dynamic Lock Control**
- **Grant Encoding and Output Register**

The request signals are first processed by the grouping and detection logic to identify active requesters. The arbitration logic then selects the appropriate requester based on token rotation, ping-pong pointers, and dynamic lock control. Finally, the selected grant signal is encoded and stored in the output register to ensure synchronized communication with the shared resource.

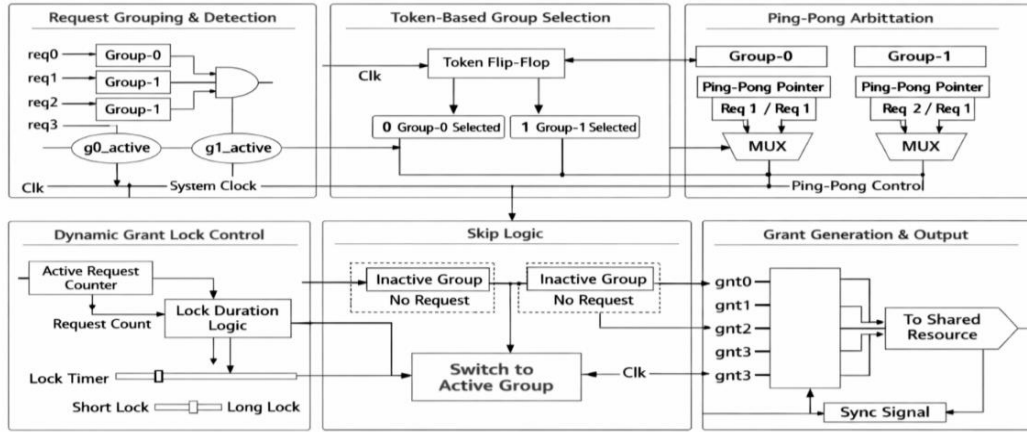


Fig 3.2 Architecture of Extended PLA

3.1.1 Request Grouping and Detection Logic

In the proposed Extended Ping Lock Arbiter (EPLA), the incoming requests are first organized using request grouping logic. The arbiter receives four request signals req [3:0], representing four different requesters trying to access the shared resource. To simplify the arbitration process, the requesters are divided into two groups. Group-0 contains requester 0 and requester 1, while Group-1 contains requester 2 and requester 3. Instead of checking all requesters at once, the arbiter first identifies which group has active requests and then performs arbitration within that group.

The activity of each group is detected using simple OR operations. If either requester in a group is active, that group is considered active. This can be represented as:

- $g0_active = req[0] \mid req[1]$
- $g1_active = req[2] \mid req[3]$

By detecting group activity first, the arbiter can avoid unnecessary checks on inactive requesters. If one group does not contain any active requests, the arbiter skips that group and moves to the other group. This grouping method reduces arbitration complexity and helps in lowering switching activity in the system.

3.1.2 Token Based Group Selection

After detecting the active groups, the arbiter selects which group should be served using a token-based selection mechanism. A single-bit token is used to indicate the currently

3.1.4 Dynamic Grant Lock Mechanism

The proposed EPLA uses a dynamic grant lock mechanism to control how long a requester holds the grant. Instead of using a fixed lock period, the lock duration is adjusted based on the number of active requests in the system.

The arbiter first calculates the total number of active requests from req [3:0]. Based on this request count, the lock duration is selected dynamically. When the number of active requests is small, the lock duration is short so that other requesters can get access quickly. When many requests are active, the lock duration is increased to reduce repeated arbitration operations.

This dynamic lock mechanism helps in reducing unnecessary switching activity and improves overall system efficiency.

3.1.5 Skip Logic

Skip logic is used to avoid unnecessary arbitration when a group does not have any active requests. If the currently selected group does not contain active requests, the arbiter automatically skips that group and switches the token to the other group. For example, if Group-0 is selected but both req [0] and req [1] are inactive, the arbiter immediately moves to Group-1 if it has active requests. Similarly, if Group-1 is inactive, the arbiter switches back to Group-0. This mechanism prevents the arbiter from wasting cycles on inactive groups and helps in reducing switching activity and power consumption.

CHAPTER-4

IMPLEMENTATION OF Extended PLA

4.1 Tool Description

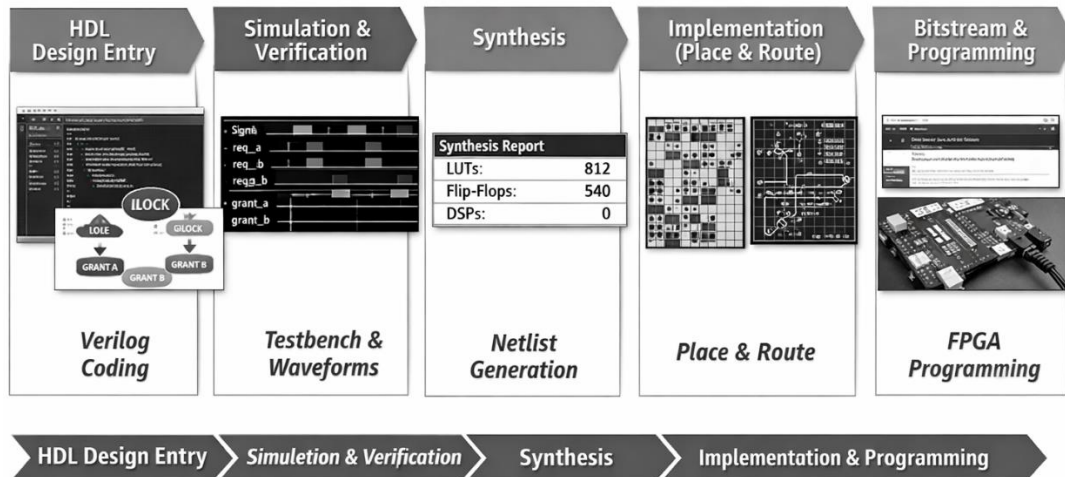


Fig 4.1 Flow chart of Tool description

HDL Design Entry

The design entry stage is the foundation of the entire Extended Ping Lock Arbiter (EPLA) project. At this stage, the arbitration logic is formally expressed in Verilog HDL using the Vivado IDE. Here, the core functionalities of the arbiter—such as request handling, grant allocation, priority resolution, and dynamic lock extension—are defined.

- The HDL code consists of modules that implement state machines, combinational logic for grant generation, and registers for storing intermediate states.
- Vivado editor provides syntax highlighting, modular design organization, and immediate error detection, which ensures the code is clear, maintainable, and free from common mistakes.
- At this point, the arbiter exists purely as a logical description—a blueprint for subsequent simulation, synthesis, and hardware implementation.

Simulation & Verification: Before moving to hardware, it's essential to validate the functional correctness of the design. This is done through simulation.

- Vivado Simulator is used to create testbenches, which simulate various traffic patterns across multiple request channels.
- Signal waveforms such as req A, req B, grant A, and grant B are observed to verify that requests are properly granted based on priority and fairness rules.

This stage ensures:

- Dynamic lock extension and release occur correctly.
- Arbitration behaves under contention without causing deadlocks or race conditions.
- The design meets the intended functional specifications before hardware mapping.

Synthesis

Once simulation verifies correctness, the design moves to synthesis, which translates the HDL code into a gate-level netlist compatible with FPGA hardware.

- Vivado synthesizer optimizes the design to reduce LUT usage, flip-flop count, and propagation delays.
- For EPLA, synthesis shows:
- LUTs used for combinational logic that calculates grants.
- Flip-flops used for state storage in the finite state machine.
- No DSP blocks were required, demonstrating efficient resource utilization.
- Logic optimization ensures minimal critical paths, which is crucial for high-speed arbitration decisions.

Implementation (Place & Route)

In the implementation stage, the synthesized netlist is mapped to the physical FPGA fabric. This involves two main steps:

1. Placement: Assigning physical locations to logic elements so that related modules are close together to minimize interconnect delay.
2. Routing: Establishing the actual connections (signal paths) between these elements while meeting timing constraints.
 - This stage is crucial for high-speed performance, especially under heavy traffic conditions where arbitration must happen every clock cycle.
 - Timing analysis ensures that no setup or hold violations occur, guaranteeing reliable operation of the arbiter in the FPGA clock domain.

Bitstream & Programming

The final stage converts the fully implemented design into a bitstream (.bit) file, which is used to program the FPGA.

- The FPGA executes the EPLA logic in real time, allowing observation of dynamic lock behaviour under actual traffic workloads.
- Hardware-level testing validates:
 - Real-time grant allocation.
 - Correct lock extension and release.
 - System performance under stress conditions.
 - Debugging can be done using Vivado ILA (Integrated Logic Analyzer), which monitors internal signals inside the FPGA.

4.2 Verilog Implementation of EPLA

The proposed Extended Ping Lock Arbiter (EPLA) is implemented using Verilog Hardware Description Language (HDL). The design mainly consists of two modules: the top module and the EPLA arbiter module. The top module acts as an interface between the external inputs and the arbiter, while the arbiter module performs the complete arbitration process.

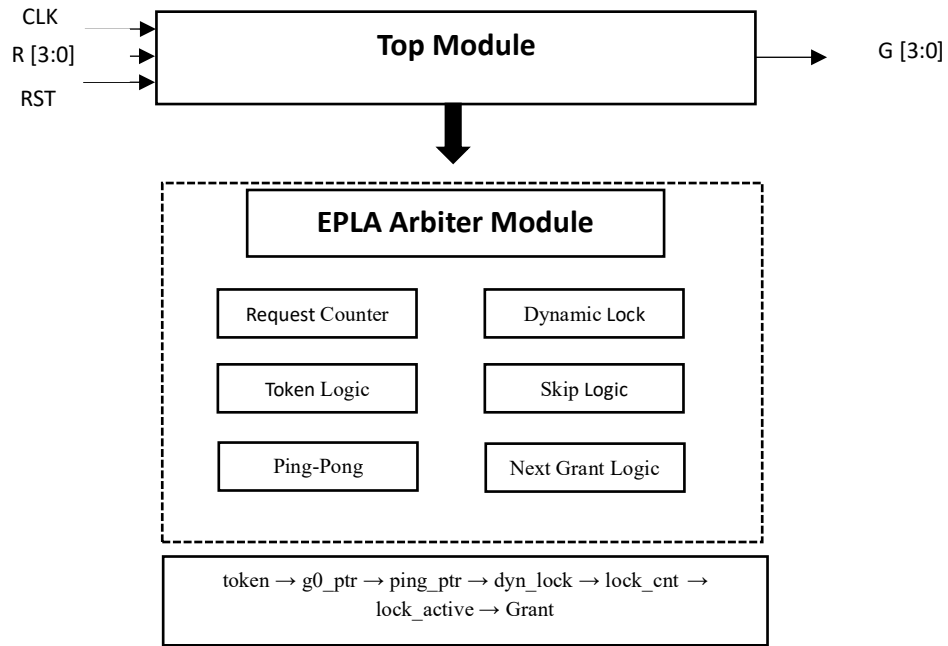


Fig 4.2 Flow of Verilog Implementation of Extended PLA

Top Module

The top module connects the system inputs such as clock, reset, and request signals to the arbiter module. It instantiates the arbiter and passes the input signals to it. The arbiter processes the requests and generates the corresponding grant outputs.

The main inputs of the top module are clk, reset, and req [3:0], which represent the system clock, reset signal, and the request signals from four requesters respectively. The output of the module is gnt [3:0], which indicates the requester that is granted access to the shared resource. The top module simply connects these inputs and outputs to the EPLA arbiter module.

Arbiter Module

The arbiter module performs the core functionality of the proposed design. It receives the request signals and determines which requester should be granted access. The arbitration process is carried out using several internal mechanisms such as group-based arbitration, token-based selection, ping-pong pointers, dynamic lock control, and skip logic.

First, the arbiter counts the number of active requests from the request inputs. Based on this count, the dynamic lock duration is determined. When the number of requests is small, the lock duration is short, and when more requests are active, the lock duration increases to reduce repeated arbitration cycles. The requests are divided into two groups. A token is used to select one group at a time. Within the selected group, a ping-pong pointer mechanism is used to choose the requester in a fair manner. After a requester is granted access, the grant is held for the duration determined by the dynamic lock counter.

Signal Description

Several internal signals are used in the implementation of the arbiter:

- **Req [3:0]** – Input request signals from four requesters
- **gnt [3:0]** – Output grant signals indicating the selected requester
- **token** – Used to select the active requester group
- **g0_ptr, g1_ptr** – Pointers used for ping-pong arbitration within groups
- **dyn_lock** – Dynamic lock value based on active request count
- **lock_cnt** – Counter used to maintain the grant for multiple cycles
- **lock_active** – Indicates whether the grant lock is currently active
- **next_gnt** – Temporary register used to store the next grant value

4.3 WORKING OF EPLA

Step 1: Reset Initialization

- When reset=1, all internal registers are cleared:
 - gnt=0000 → no requester has access.
 - token=0 → start with Group-0.
 - g0_ptr=0, g1_ptr=0 → ping-pong pointers reset.
 - Lock_cnt =0, lock_active=0 → no lock is active.
 - first_cycle=1 → special flag for initial priority.
- This ensures the arbiter starts in a clean state.

Step 2: Request Counting & Dynamic Lock Setup

- The arbiter counts active requests:

$$req_count = req[0] + req[1] + req[2] + req[3]$$

- Based on this, dyn_lock is set:
 - 1 cycle if ≤ 1 request.
 - 2 cycles if exactly 2 requests.
 - cycles if ≥ 3 requests.
 - This adaptive lock duration reduces repeated arbitration when traffic is heavy.

Step 3: Group Activity Detection

- Groups are checked:
 - $g0_active = req[0] \mid req[1]$
 - $g1_active = req[2] \mid req[3]$
 - Only active groups are considered. If one group is inactive, arbiter skips it.

Step 4: First-Cycle Priority

- On the very first cycle (first_cycle=1):
- If $req[0] = 1$, requester 0 gets immediate grant (gnt=0001).
 - Lock is activated (lock_active=1) and held for dyn_lock cycles.
 - first_cycle flag is cleared after this.
 - This ensures deterministic startup behavior.

Step 5: Lock Countdown (Grant Hold)

- While lock_active=1:
 - Arbiter holds the current grant (gnt remains stable).
 - lock_cnt decrements each cycle.
 - When lock_cnt==1, lock expires:
- Arbiter releases current grant.
- Token may switch to the other group if requests are pending.

Step 6: Normal Arbitration

- After lock release, arbiter performs arbitration:

Group-0 Arbitration (token=0):

- If req [0] is active and either pointer=0 or req [1] =0, requester 0 gets grant.
- Else if req [1] is active, requester 1 gets grant.
- Ping-pong pointer (g0_ptr) toggles after each grant.

Group-1 Arbitration (token=1):

- If req [2] is active and either pointer=0 or req [3] =0, requester 2 gets grant.
- Else if req [3] is active, requester 3 gets grant. Ping-pong pointer (g1_ptr) toggles after each grant.
- After arbitration, lock is re-activated for dyn_lock cycles.

Step 7: Skip Logic

- If the current token group has no active requests:
- Arbiter immediately switches token to the other group if it has requests.
- This prevents wasted cycles and reduces switching activity.

Step 8: Grant Update

- The selected grant is stored in next_gnt.
- If gnt_en (any bit of next_gnt is high), arbiter updates gnt [3:0] synchronously with the clock.
- Only one bit of gnt is active at a time → ensures mutual exclusion.

4.4 METHODOLOGY

Group Activity Detection

$$G_0 = \begin{cases} 1, & \text{if } Req_0 = 1 \text{ or } Req_1 = 1 \\ 0, & \text{Otherwise} \end{cases} \quad (1)$$

$$G_1 = \begin{cases} 1, & \text{if } Req_2 = 1 \text{ or } Req_3 = 1 \\ 0, & \text{Otherwise} \end{cases} \quad (2)$$

Outer Group Selection

$$Group_{selection} = \begin{cases} 0, & \text{When } Token=0 \text{ and } G_0=1 \\ 1, & \text{When } Token=1 \text{ and } G_1=1 \end{cases} \quad (3)$$

Inner Group Selection

$$Grant_j = \begin{cases} 1, & \text{If the requester j is active} \\ 0 & \text{Otherwise} \end{cases} \quad (4)$$

Request Load Estimation

$$Req_{count} = \sum_{j=0}^3 Req_j \quad (5)$$

Dynamic Grant Lock

$$Dyn_LOCK = \begin{cases} 1, & Req_{count} \leq 1 \\ 2, & Req_{count} = 2 \\ 3, & Req_{count} \geq 3 \end{cases} \quad (6)$$

$$Lock = \begin{cases} 1, & Lock\ Counter > 0 \\ 0, & Lock\ Counter = 0 \end{cases} \quad (7)$$

$$Lock\ Counter_{next} = Lock\ Counter - 1$$

Token Updation

$$Token_{next} = (C^- \cdot O) + Token \cdot (C^- \cdot O + C \cdot O^-) \quad (8)$$

Where c= current group active

Arbitration Delay

$$T_{arb} = T_{group} + T_{request} \quad (9)$$

4.5 RTL Schematic

After completing the Verilog design, the proposed EPLA is synthesized using Xilinx Vivado. During synthesis, Vivado generates the RTL (Register Transfer Level) schematic, which represents the logical structure of the design. The RTL schematic shows how the different modules and internal signals are interconnected. It includes the top module, the EPLA arbiter module, and the internal logic blocks such as registers, combinational logic, and control signals used in the arbitration process.

In the schematic, the input signals `clk`, `reset`, and `req [3:0]` are connected to the arbiter module. Inside the arbiter, logic blocks implement the request grouping, token-based group selection, ping-pong pointer control, and dynamic lock mechanism. The generated grant signals `gnt [3:0]` are then connected to the output through registers to ensure synchronous operation with the system clock. The RTL schematic helps verify that the designed logic correctly follows the intended architecture. It also provides a visual representation of the internal connections and confirms that the arbitration logic, grouping mechanism, and grant generation are properly implemented in the hardware design.

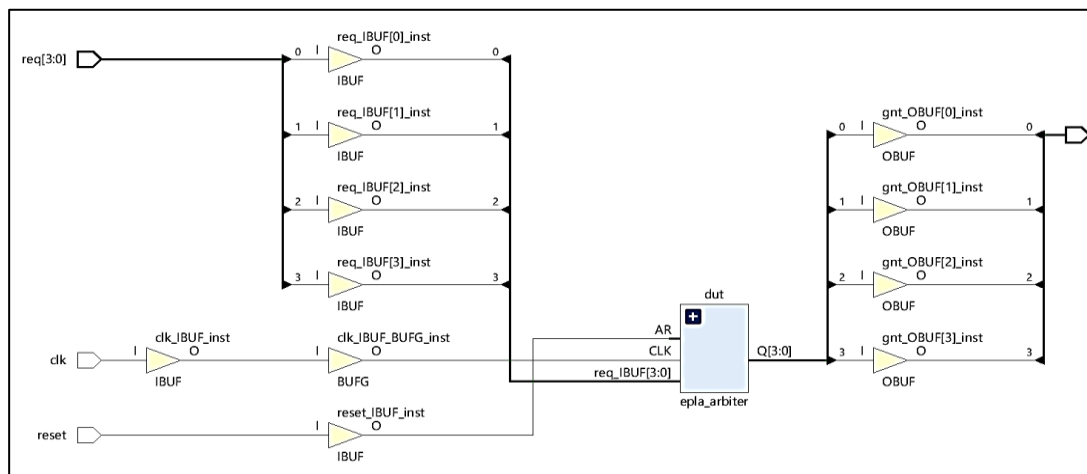


Fig 4.3 Schematic Diagram of Extended PLA

The implementation schematic of the proposed EPLA is represented in Fig. The clock is then forwarded to input as well as global buffers to enable synchronization. The request inputs `Req [3:0]`, as well as the reset signal, are connected to input buffers to

ensure that they are received by the arbiter core correctly. The EPLA module and the resulting grant signals Gnt [3:0] are connected to output buffers before being forwarded to the output ports. At a time, only one grant signal is asserted to ensure proper arbitration and proper allocation of resources in the FPGA module.

4.6 FPGA Implementation

The hardware implementation of the proposed design is carried out using the **Nexys-4 DDR FPGA development board**. This board is widely used for digital system design and FPGA prototyping because it provides a reliable platform with multiple input and output peripherals. The Nexys-4 DDR board is built around the **Artix-7 FPGA** from **Xilinx**. The specific FPGA device used on this board is **XC7A100T-1CSG324C**. Here, **XC7A100T** represents the FPGA device family and logic capacity, **CSG324** indicates the package type with 324 pins, and **-1** denotes the speed grade of the FPGA.

The Artix-7 FPGA belongs to the **7-series FPGA family**, which is designed for high performance and low power consumption. It provides sufficient logic resources, flip-flops, and routing capabilities required for implementing complex digital circuits such as arbiters and other control architectures. In addition to the FPGA device, the Nexys-4 DDR board includes various onboard peripherals such as **slide switches, push buttons, LEDs, seven-segment displays, clock sources, and DDR memory**, which make it suitable for implementing and testing digital hardware designs. These features allow designers to easily interact with the FPGA and observe the behaviour of the implemented system.

CHAPTER-5

RESULTS AND SIMULATION

5.1 Software Used – Xilinx Vivado – 2020.1

In this project, the design and implementation of the proposed Extended Ping Lock Arbiter (EPLA) is carried out using **Xilinx Vivado 2020.1**. Vivado is a powerful design tool developed for creating and implementing digital circuits on Xilinx FPGA devices.

This project uses Vivado to write, simulate, and implement the Verilog-based arbiter design. The tool provides a complete development environment where the hardware description code can be designed, tested, and converted into a form that can run on FPGA hardware.

5.2 Simulations and Analysis

Case 1: Single Request (req = 0001)

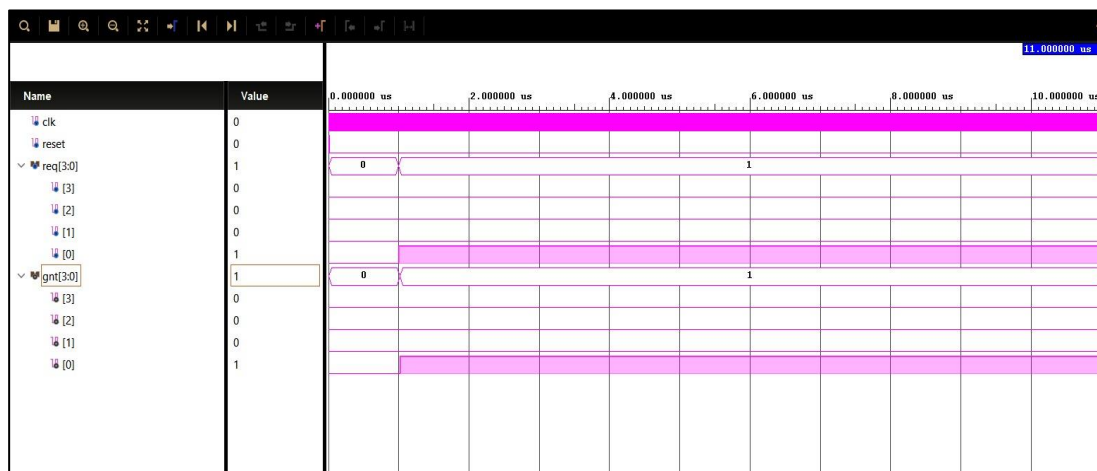


Fig 5.1 Simulation result for single requester arbitration

In this case, only Requester 0 generates a request signal while the remaining requesters are inactive. The input combination is req = 0001. The arbiter first counts the number of active requests. Since only one request is present, the dynamic lock value becomes 1 cycle. Therefore, the arbiter grants access to the requester for only one clock cycle. When the clock signal is applied, the arbiter checks the request and directly grants access to Requester 0 because no other requester is competing for the shared resource.

Observed Output: Clock Cycle 1 $\rightarrow$ gnt = 0001

After the lock duration expires, the arbiter checks again for new requests. Since no other requests are present, the same requester continues to receive the grant whenever the request remains active.

Case 2: Requests from Different Groups (req = 1010)

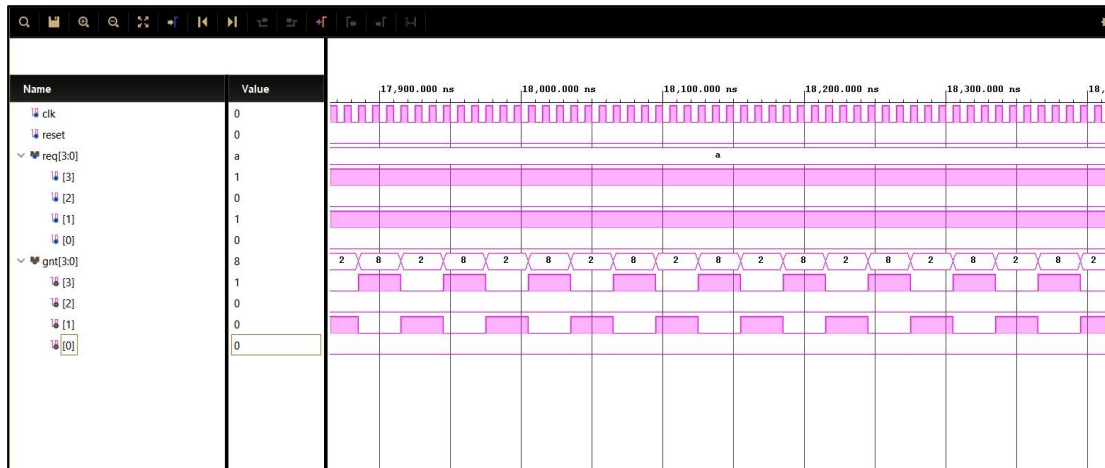


Fig 5.2 Simulation result for dual requester arbitration

In this case, Requester 1 and Requester 3 generate request signals while the other requesters remain inactive. The input combination is req = 1010. Here, two requests are active. According to the dynamic lock logic, when two requests are present, the dynamic lock value becomes 2 cycles. This means the grant signal remains active for two clock cycles before switching.

The requesters belong to different groups:

- Group 0: Requester 0 and Requester 1
- Group 1: Requester 2 and Requester 3

For the input 1010, Requester 1 belongs to Group 0 and Requester 3 belongs to Group 1. If the token initially points to Group 0, the arbiter grants access to Requester 1.

Observed Output:

Clock Cycle 1 $\rightarrow$ gnt = 0010

Clock Cycle 2 $\rightarrow$ gnt = 0010

After the lock duration expires, the token switches to Group 1, and the arbiter grants access to Requester 3.

Clock Cycle 3 → gnt = 1000

Clock Cycle 4 → gnt = 1000

Case 3: Multiple Requests (req = 1110)

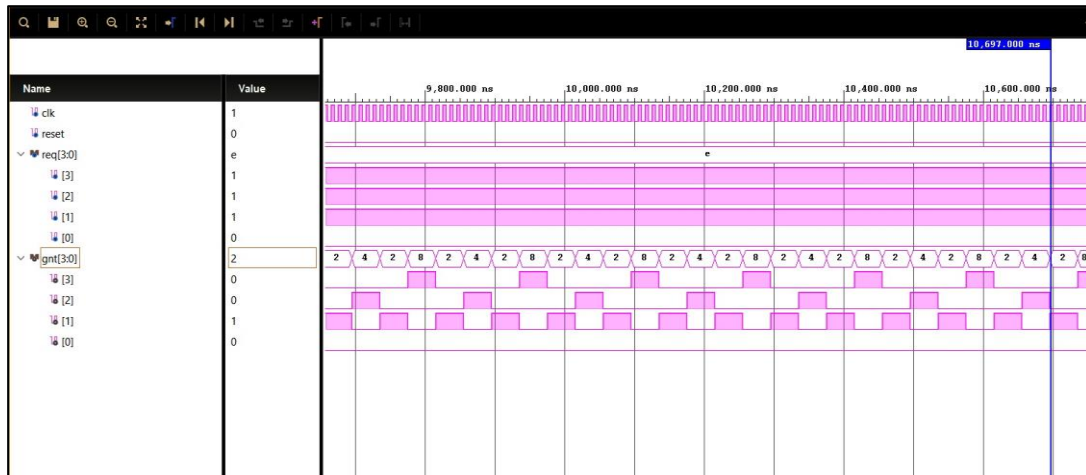


Fig 5.3 Simulation result for multiple requester arbitration

In this case, Requester 1, Requester 2, and Requester 3 generate request signals while Requester 0 remains inactive. The input combination is req = 1110. Here, three requests are active. According to the dynamic lock logic, when three or more requests are present, the dynamic lock value becomes 3 cycles. This means the grant signal will remain active for three clock cycles before switching.

The requesters are divided into two groups:

- Group 0: Requester 0 and Requester 1
- Group 1: Requester 2 and Requester 3

In this input condition, Requester 1 is active in Group 0, while Requester 2 and Requester 3 are active in Group 1. If the token initially points to Group 0, the arbiter first grants access to Requester 1.

Observed Output:

Clock Cycle 1 → gnt = 0010

Clock Cycle 2 → gnt = 0010

Clock Cycle 3 → gnt = 0010

After the lock duration expires, the token switches to Group 1. Within this group, the arbiter uses the ping-pong mechanism to alternate between Requester 2 and Requester 3.

Clock Cycle 4 → gnt = 0100

Clock Cycle 5 → gnt = 0100

Clock Cycle 6 → gnt = 0100

Next arbitration cycle:

Clock Cycle 7 → gnt = 1000

Case 4: All Requests Active (req = 1111)

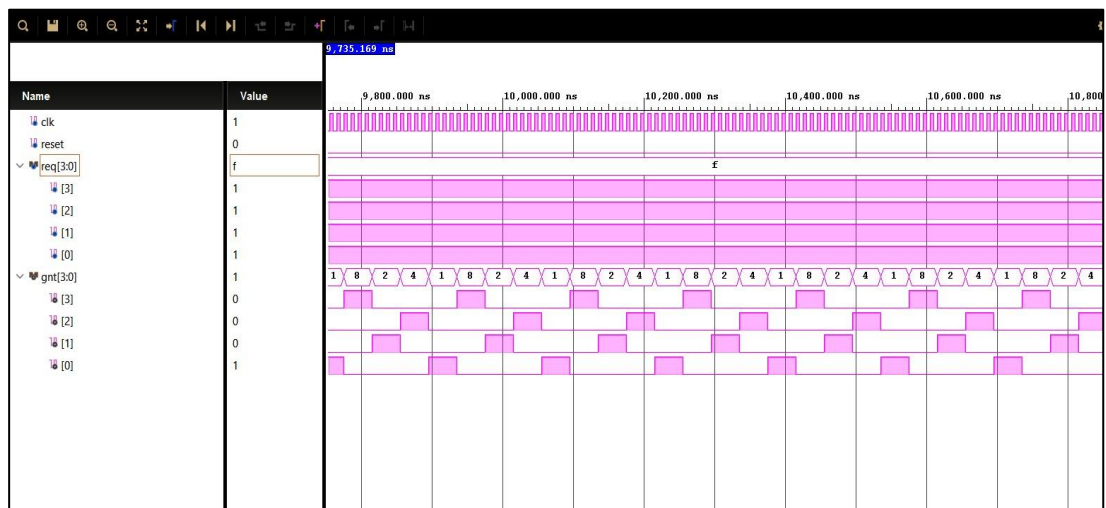


Fig 5.4 Simulation result for multiple requester arbitration

In this case, all four requesters generate request signals simultaneously. The input combination is req = 1111, which means every requester is requesting access to the shared resource at the same time.

The arbiter first counts the number of active requests. Since four requests are active, the dynamic lock value becomes 3 cycles according to the dynamic lock logic.

Therefore, the grant signal remains active for three clock cycles before switching to another requester.

The requesters are divided into two groups:

- Group 0: Requester 0 and Requester 1
- Group 1: Requester 2 and Requester 3

If the token initially points to Group 0, the arbiter selects a requester from this group using the ping-pong arbitration mechanism.

Observed Output:

Clock Cycle 1 → gnt = 0001

Clock Cycle 2 → gnt = 0001

Clock Cycle 3 → gnt = 0001

After the lock duration expires, the arbiter switches to the next requester in the same group due to the ping-pong pointer.

Clock Cycle 4 → gnt = 0010

Clock Cycle 5 → gnt = 0010

Clock Cycle 6 → gnt = 0010

After serving Group 0, the token switches to Group 1, and the arbiter grants access to the requesters in that group.

Clock Cycle 7 → gnt = 0100

Clock Cycle 8 → gnt = 0100

Clock Cycle 9 → gnt = 0100

Next arbitration cycle:

Clock Cycle 10 → gnt = 1000

Table 3: Arbitration Results for Different Request Combinations

Case	Request Input (req [3:0])	Active Requesters	Dynamic Lock (Cycles)	Grant Output Sequence (gnt [3:0])
Case 1	0001	R0	1	0001
Case 2	1010	R1, R3	2	0010 → 0010 → 1000 → 1000
Case 3	1110	R1, R2, R3	3	0010 → 0010 → 0010 → 0100 → 0100 → 0100 → 1000
Case 4	1111	R0, R1, R2, R3	3	0001 → 0001 → 0001 → 0010 → 0010 → 0010 → 0100 → 0100 → 0100 → 1000

Reason for Extra Cycle in Output

During observation of the output sequence, an extra clock cycle may appear before the next requester receives the grant. This occurs due to the lock release and token switching process in the arbiter. When the dynamic lock counter reaches its final value, the arbiter first disables the current lock and updates the internal control signals such as the token and ping-pong pointer.

Since these updates occur sequentially inside the clocked always block, the arbiter requires one additional clock cycle to complete the transition before issuing the next grant signal. As a result, the next requester receives the grant in the following clock cycle.

5.2.1 POWER ANALYSIS

The analysis shows that the total power consumption of the integrated design is **0.486 W**, which consists of both dynamic and static power components. Dynamic power contributes the major portion with **0.394 W (81%)**, while static power accounts for **0.092 W (19%)** due to device leakage. Within the dynamic power category, **I/O power dominates the overall power consumption**, contributing **0.269 W (69%)**, followed by **signal power of 0.068 W (17%)** and **logic power of 0.056 W (14%)**. These results

indicate that the overall power consumption is mainly influenced by FPGA input/output switching activity, whereas the internal logic and signal power remain comparatively low, demonstrating efficient resource utilization of the proposed EPLA architecture

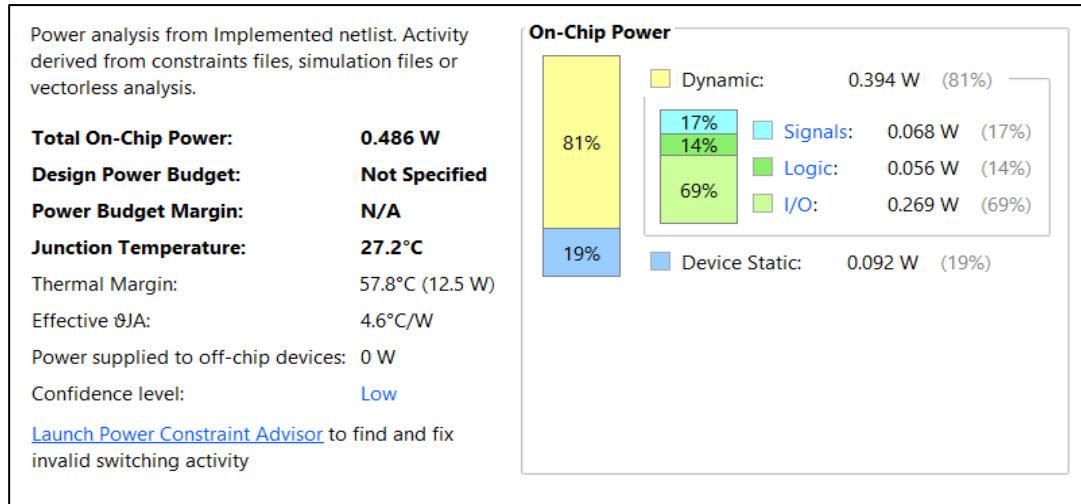


Fig 5.5 Power Report of Extended PLA

5.2.2 TIMING ANALYSIS

Timing Report				
Slack:	inf			
Source:	dut/gnt_reg[0]/C (rising edge-triggered cell FDCE)			
Destination:	gnt[0] (output port)			
Path Group:	(none)			
Path Type:	Max at Slow Process Corner			
Data Path Delay:	4.837ns (logic 3.104ns (64.182%) route 1.732ns (35.818%))			
Logic Levels:	2 (FDCE=1 OBUF=1)			
Location	Delay type	Incr (ns)	Path (ns)	Netlist Resource(s)
SLICE_X0Y53	FDCE	0.000	0.000	r dut/gnt_reg[0]/C
SLICE_X0Y53	FDCE (Prop_fdce_C_Q)	0.456	0.456	r dut/gnt_reg[0]/Q
	net (fo=1, routed)	1.732	2.188	gnt_OBUF[0]
R10	OBUF (Prop_obuf_I_O)	2.648	4.837	r gnt_OBUF[0]_inst/O
	net (fo=0)	0.000	4.837	gnt[0]
R10				r gnt[0] (OUT)

Fig 5.6 Timing Report of Extended PLA

The results indicate that the designed system satisfies all the required timing constraints with positive slack values, confirming that the circuit operates correctly within the specified clock period. The analysis shows that the maximum data path delay is 4.837 ns, which represents the longest delay encountered during data transfer from the grant

register to the output. This delay consists of 64.18% logic delay and 35.82% routing delay within the FPGA. The relatively small delay is mainly due to the simple arbitration architecture, where the data propagation path involves only two levels of logic before reaching the output. These results demonstrate that the proposed design achieves efficient timing performance and can operate reliably at higher clock frequencies.

5.3 Hardware Verification

The hardware verification of the proposed EPLA is performed on the Nexys-4 DDR FPGA board. The request inputs req [3:0] are applied using switches SW1–SW4, while the reset signal is provided through SW0. The clock is given using the push button BTNC. Initially, the system is reset using SW0. The request inputs req [3:0] are then applied using switches SW1–SW4, while the clock is given using the push button BTNC. Each clock press performs one arbitration cycle. Different request combinations are tested to verify the behaviour of the arbiter.

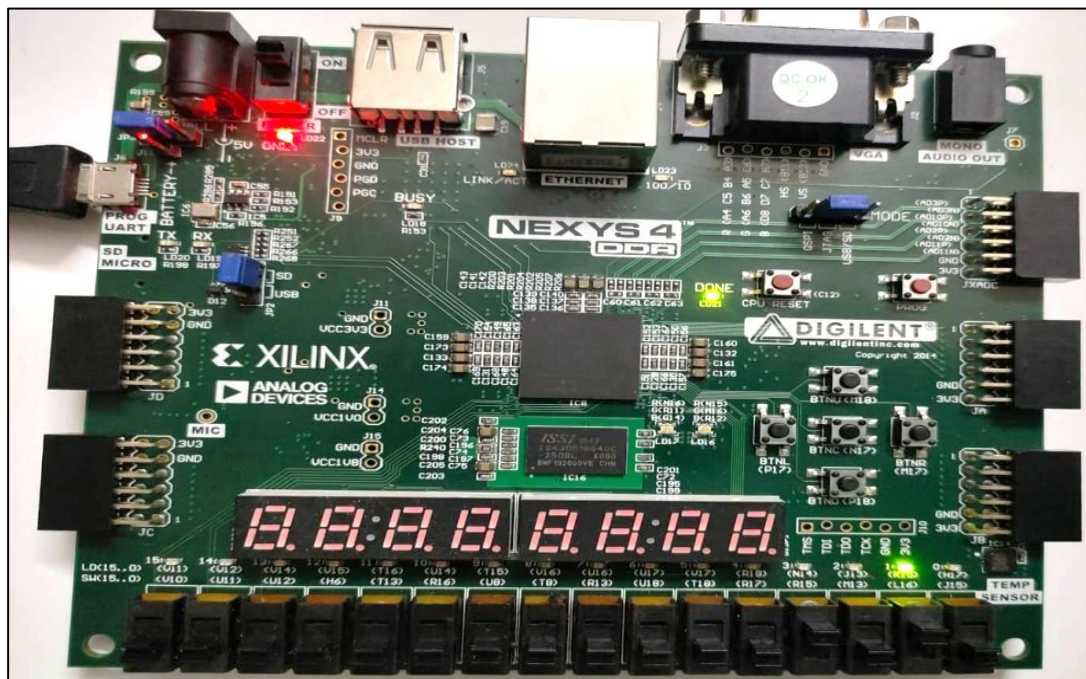


Fig 5.7 FPGA Verification of Extended PLA

Single Request: If only one request is active (for example SW1 ON), the corresponding LED (LED1) turns ON, indicating that the requester is granted access.

Two Requests in Same Group:

If requests from the same group are active (for example SW1 and SW2), the grant alternates between them using the ping-pong mechanism. This can be observed as LEDs switching between LED1 and LED2 for each clock pulse.

Requests from Different Groups:

If requests are present in both groups (for example SW1 and SW3), the arbiter uses the token mechanism to select one group first and then switches to the other group after the lock duration.

Multiple Requests:

When three or four requests are active, the arbiter grants access sequentially based on the dynamic lock and token switching, which can be observed as LEDs rotating among the active requesters.

5.4 Comparative Analysis

Table 4: Comparative Analysis

S. No	Comparison	Dynamic Power (W)	Total ON-Chip Power (W)	Switching Activity
1	Proposed Extended Ping Lock Arbiter	0.394	0.486	Reduced
2	Existing (Enhanced ping lock arbiter)	1.089	1.172	High
3	% change w.r.t Existing	63.8%	58.5%	Significantly Reduced

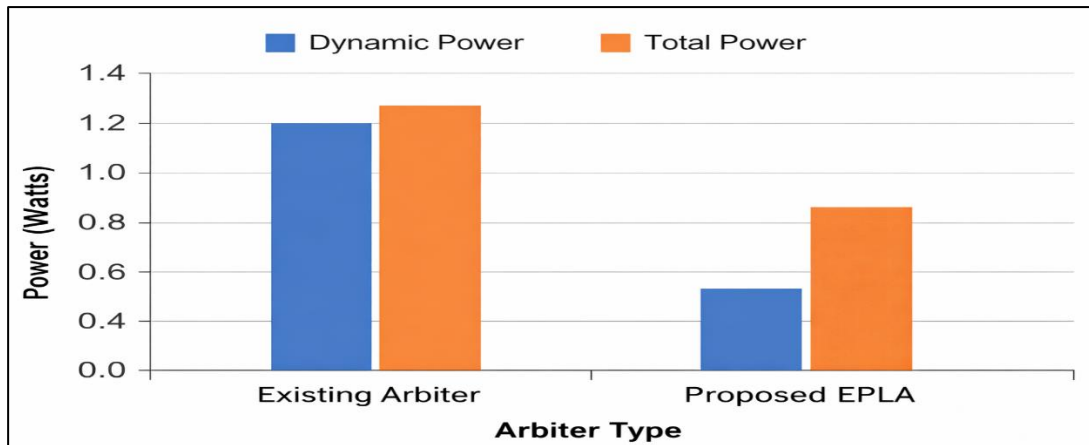


Fig 5.8 Power Distribution of Extended PLA

Switching activity refers to the number of signal transitions ($0 \rightarrow 1$ or $1 \rightarrow 0$) occurring within the FPGA fabric during operation. These transitions cause capacitive charging and discharging, which directly contributes to dynamic power consumption.

Existing Enhanced Ping Lock Arbiter:

- Exhibits high switching activity due to frequent toggling of grant signals across multiple requesters.
- No dynamic locking mechanism $\rightarrow$ grant signals change every cycle $\rightarrow$ more transitions.
- Result: Dynamic Power = 1.089 W

Proposed Extended Ping Lock Arbiter (EPLA):

- Uses dynamic locking and ping-pong arbitration to minimize unnecessary transitions.
- Grant signal remains stable for multiple cycles $\rightarrow$ fewer toggles.
- Result: Dynamic Power = 0.394 W
- Switching activity is significantly reduced, leading to 63.8% lower dynamic power.

Dynamic Power Consumption:

Dynamic power mainly depends on switching activity within the FPGA. The existing Enhanced Ping Lock Arbiter consumes 1.089 W, whereas the proposed EPLA reduces it to 0.394 W, achieving about 63.8% reduction. This improvement is mainly due to the dynamic locking mechanism, which minimizes unnecessary signal transitions and reduces power consumption.

Total On-Chip Power:

Total on-chip power includes both dynamic and static power components. The existing design consumes 1.172 W, while the proposed EPLA requires only 0.486 W, resulting in about 58.5% reduction. Lower on-chip power improves energy efficiency and helps maintain better thermal stability in FPGA devices.

Switching Activity:

Switching activity significantly affects dynamic power consumption. The existing arbiter shows higher switching activity due to frequent signal transitions. In contrast, the proposed EPLA reduces unnecessary toggling by using controlled arbitration and dynamic locking, leading to more efficient FPGA operation.

CONCLUSION

The proposed adaptive lock control mechanism dynamically adjusts the grant duration based on the activity level of requesters, thereby improving the overall efficiency of arbitration in shared resource systems. By incorporating dynamic lock control along with grouping and skip logic, the design effectively minimizes unnecessary arbitration operations.

The grouping mechanism, combined with skip logic, enables the arbiter to ignore inactive requesters or groups, significantly reducing redundant processing and improving system performance. This results in a substantial reduction in switching activity by 63.8%, which directly contributes to lower dynamic power consumption. Furthermore, When compared to the conventional Enhanced Ping Lock Arbiter, the proposed design demonstrates a remarkable decrease in internal transitions. Consequently, the total on-chip power consumption is reduced by 58.5%, making the system more energy-efficient and suitable for modern low-power applications.

Overall, the proposed arbiter offers an optimized, scalable, and power-efficient solution for resource sharing in VLSI and System-on-Chip (SoC) designs. Its ability to adapt dynamically to varying request patterns makes it highly effective for use in shared bus architectures and Network-on-Chip (NoC) environments.

FUTURE SCOPE

- The proposed EPLA can be extended to support a larger number of requesters, making it suitable for highly complex System-on-Chip (SoC) and Network-on-Chip (NoC) architectures. This enhancement would allow the arbiter to efficiently handle increased communication demands in large-scale systems.
- Scalability Enhancement: Extend the EPLA to support a larger number of requesters, making it suitable for complex SoC and NoC systems.
- Hierarchical Grouping: Introduce multi-level grouping of requesters to reduce arbitration complexity and improve efficiency under heavy traffic.
- Adaptive Arbitration Mechanism: Enhance token rotation using priority-based or intelligent scheduling to improve fairness and reduce waiting time.

REFERENCES

- [1] P. Karanpuria, “Fair chance round robin arbiter,” *Int. J. Comput. Appl.*, vol. 81, no. 14, pp. 36–42, Nov. 2013.
- [2] R. Shashidhar, S. N. Sujay, and G. S. Pavan, “Implementation of bus arbiter using round robin scheme,” *Int. J. Innov. Res. Sci., Eng. Technol.*, vol. 3, no. 7, pp. 14937–14944, Jul. 2014.
- [3] A. S. N. Asha and B. A. U. Bindu, “Design and implementation of index-based round robin arbiter for NoC routers using FPGA,” *Int. J. Innov. Res. Sci., Eng. Technol.*, vol. 5, no. 5, pp. 7448–7453, May 2016, doi: 10.15680/IJIRSET.2016.0505141.
- [4] A. Mishra, “Design of a round robin bus arbiter using SystemVerilog,” *Int. Res. J. Eng. Technol. (IRJET)*, vol. 7, no. 1, pp. 1492–1494, Jan. 2020.
- [5] N. L. Venkataraman et al., “Design of matrix, distributive round robin, ping pong and enhanced ping lock arbiter for shared resources systems,” *Indones. J. Electr. Eng. Comput. Sci.*, vol. 32, no. 3, pp. 1337–1345, Dec. 2023, doi: 10.11591/ijeecs.v32.i3.pp1337-1345.
- [6] N. L. Venkataraman et al., “Design and implementation of different arbiters for shared resource systems,” *Theory Appl. Eng. Res.*, vol. 4, 2023, doi: 10.9734/bpi/taer/v4/2779G.
- [7] N. M. and V. Shylaja, “FPGA implementation of priority-arbiter based router design for NoC systems,” *Int. J. Adv. Res. Eng. Technol.*, vol. 10, no. 2, pp. 509–516, Mar.–Apr. 2019.
- [8] T. Kavitha, S. Shiyamala, and P. Nagarajan, “FPGA implementation of arbiters algorithm for network-on-chip routers,” *ARPJ. Eng. Appl. Sci.*, vol. 11, no. 19, pp. 11451–11456, 2016.
- [9] M. Abdelrasoul, M. Ragab, and V. Goulart, “Evaluation of the scalability of round robin arbiters for NoC routers on FPGA,” in *Proc. IEEE MCSoc*, 2013.
- [10] B. Debbarma and J. Debbarma, “Clock-gated round robin arbiter for NoC router,” *MATTER: Int. J. Sci. Technol.*, vol. 3, no. 3, pp. 113–124, Nov. 2017.

- [11] R. S. Shashidhar and G. S. Pavan, "FPGA implementation of a bus arbiter for shared resources," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 5, no. 6, pp. 123–128, Jun. 2016.
- [12] N. M. and V. Shylaja, "Design and implementation of priority arbiter based router," *Int. J. Adv. Res. Eng. Technol.*, vol. 10, no. 2, pp. 509–516, 2019.
- [13] T. Kavitha, S. Shiyamala, and P. Nagarajan, "FPGA implementation of arbiters algorithm for network-on-chip routers," *ARPN J. Eng. Appl. Sci.*, vol. 11, no. 19, pp. 11451–11456, 2016.
- [14] A. S. Tigadi and H. Guhilot, "Design of an arbiter for two systems accessing a single DDR3 memory on a reconfigurable platform," *Int. J. Inf. Eng. Electron. Bus.*, vol. 10, no. 6, pp. 14–20, Nov. 2018, doi: 10.5815/ijieeb.2018.06.02.
- [15] A. Paul et al., "Reconfigurable parallel architecture of high-speed round robin arbiter," *arXiv preprint*, Mar. 2020.
- [16] Adrika Mohanty and Chiranjeevi G N., "Comparative Analysis of Round Robin Arbiter and Weighted Round Robin Arbiter," pp. 26-27 July 2024, Publisher: IEEE, DOI: 10.1109/APCIT62007.2024.10673654.
- [17] Anuradha Shenoy, Anika Kirana Venkat, Adithya S. Chakravarty, Smitha Gayathri D. "Implementation of Fair Chance Round Robin Arbiter and its Comparative Analysis," pp: 26-27 September 2025, Publisher: IEEE, DOI: 10.1109/ICWITE64848.2025.11307144.
- [18] K. Yoghigoe, K.J. Christensen, A. Roginsky, "Design of a high-speed overlapped round robin (ORR) arbiter," pp: 20-24 October 2003, Publisher: IEEE, DOI: 10.1109/LCN.2003.1243193.
- [19] Khaled A. Helal, Sameh Attia, Tawfik Ismail, Hassan Mostafa, "Priority-select arbiter: An efficient round-robin arbiter," pp: 07-10 June 2015, DOI: 10.1109/NEWCAS.2015.7182062.
- [20] J. Reed, N. Manjikian, "A dual round-robin arbiter for split-transaction buses in system-on-chip implementations," pp: 2004, IEEE Cat No. 04CH37513)

- [21] S. Q. Zheng, Mei Yang, J. Blanton, P. Golla, D. Verchere “A simple and fast parallel round-robin arbiter for high-speed switch control and scheduling,” pp: 2002, DOI: 10.1109/MWSCAS.2002.1186951.
- [22] Jagrati Gupta, Nidhi Goel, “Efficient bus arbitration protocol for SoC Design,” pp: 2015, Publisher: IEEE
- [23] D. Shanthi, R. Amutha, “Design of efficient on-chip communication architecture in MpSoC,” pp: 2006, Publisher: IEEE, DOI: 10.1109/ICNICONSMCL.2006.168
- [24] Abhilash L Bedre, V. Nithish Kumar, “A Hybrid arbiter to accelerate performance of high speed SoC,” pp: 2017, Publisher: IEEE
- [25] Subhah Gogula, V. Damodaran, “Design of a VLSI Router For the faster Data Transmission Using Buffer,” pp: 21-22 April 2023, Publisher: IEEE, DOI: 10.1109/ICSTSN57873.2023.101568.

RESUME

Name of the candidate: MASA RUPA MANI LATHA

Regd No: Y22AEC509

Address for correspondence: 4-8-21/a, Akbarpet, Bapatla Mandal,
Bapatla District, Andhra Pradesh

Email id: rupamanilatha9@gmail.com

Phone: 9959452089

Academic Record

Course	College	Year of Passing	% Marks
S.S.C	Little Flower High School	2020	10 CGPA
Intermediate	Sri Sai Junior College	2020-2022	97%
B. Tech	Bapatla Engineering College	2022-2026	9.4 CGPA

Academic Achievements

Projects:

Electronic Voting Machine – Arduino

- Designed and implemented a secure electronic voting system using Arduino Uno.
- Enabled real-time vote counting with accurate results, reducing manual errors.

Personal Portfolio Website

- Developed a responsive portfolio website using HTML5, CSS3, JavaScript(basic).
- Focused on UI/UX, interactive design, and cross-browser compatibility.

Internships:

- **Embedded Systems Internship – BIST Pvt. Ltd.:** Designed embedded modules using Arduino.
- **Frontend Development Internship – Talentshine Pvt. Ltd.:** Built responsive websites with HTML, CSS.

Certifications:

- **Physical Design Workshop – VLSI GURU**

- **Frontend Workshop – Kalki Tech**
- **Cloud Computing - NPTEL JAN 2025**
- **IOT - NPTEL JAN 2025**

RESUME

Name of the candidate: MARUBOINA PAVAN KRISHNA

Regd No: L23AEC599

Address for correspondence: 1-203, Janakiramapuram, Bantumilli
Mandal, Krishna District, Andhra Pradesh

Email id: pavanmaruboinao09@gmail.com

Phone: 8074140668

Academic Record

Course	College	Year of Passing	% Marks
S.S.C	Margadarsi (EM & TM) High School	2020	10 CGPA
Intermediate	A.A.N.M & V.V.R.S.R Seshadri Rao Polytechnic College	2020-2023	83%
B. Tech	Bapatla Engineering College	2023-2026	6.9 CGPA

Academic Achievements

Projects:

- Smart Networking (Based on IOT)
- Wireless power Transmission (Based on IOT)
- Smart Parking Using Embedded Systems
- Auto Temperature Controller

Internships:

- Embedded Systems – Vision INFO TECH
- VLSI Design – Skill Dzure India
- VLSI Design for Testability-BIST Technologies

Certifications:

- FORNTEND VLSI Design – KALKI TECH
- Backend VLSI Design – VLSI GURU

- Cloud Computing-NPTEL
- The Joy of Computing Using Python – NPTEL

RESUME

Name of the candidate: MIRAPALA KANCHANA

Regd No: Y22AEC514

Address for correspondence: Sriramapuram, Venkatapuram post, Tadapatri, Jangampeta Kotauratla, Visakhapatnam, Andhra Pradesh 531085

Email id: kanchanamirapala366@gmail.com

Phone: 9100970924

Academic Record:

Course	College	Year of Passing	% Marks
S.S.C	Loyola English medium School	2020	89.3% CGPA
Intermediate	Narayana Junior College	2020-2022	70%
B. Tech	Bapatla Engineering College	2022-2026	7.0 CGPA

Academic Achievements

Projects:

- VLSI Design for RFID

Internships:

- VLSI FRONT END – Skill Dzire India
- VLSI Design for Testability-BIST Technologies

Certifications:

- FORNTEND VLSI Design – KALKI TECH
- Backend VLSI Design – VLSI GURU
- Cloud Computing-NPTEL
- Privacy and security - NPTEL

RESUME

Name of the candidate: OGIBOINA MANIKANTA

Regd No: Y22AEC529

Address for correspondence: 1-10-52, 5th ward, Railpeta,
Bapatla, Bapatla Mandal, BAPATLA District, Andhra Pradesh

Email id: ovrlu.7@gmail.com

Phone: 9948848016

Academic Record:

Course	College	Year of Passing	% Marks
S.S.C	Little Angels High School	2020	90%
Intermediate	Sri Chaithanya Junior College	2020-2022	65%
B. Tech	Bapatla Engineering College	2022-2026	4 CGPA

Academic Achievements

Projects:

- Smart Networking (Based on IOT)
- Wireless power Transmission (Based on IOT)
- Auto Temperature Controller

Internships:

1. VLSI Design – Skill Dzire India
2. VLSI Design for Testability-BIST Technologies

Certifications:

- FORNTEND VLSI Design – KALKI TECH
- Backend VLSI Design – VLSI GURU

PUBLICATION DETAILS

(12) PATENT APPLICATION PUBLICATION (19) INDIA (22) Date of filing of Application :21/02/2026	(21) Application No.202641020393 A (43) Publication Date : 06/03/2026
(54) Title of the invention : Dynamic Group-Based Ping Lock Arbiter with Skip Logic for 4 Requesters	
(51) International classification (31) Priority Document No (32) Priority Date (33) Name of priority country (86) International Application No Filing Date (87) International Publication No (61) Patent of Addition to Application Number Filing Date (62) Divisional to Application Number Filing Date	(71)Name of Applicant : 1)Dr. P. SURENDRA KUMAR Address of Applicant :Associate Professor, ECE Department, Bapatla Engineering College, Bapatla Andhra Pradesh India 2)Dr. Ch.V.M.S.N. Pavan Kumar 3)Dr. Kommalapati Rajesh 4)M. Rupa Mani Latha 5)M. Pavan Krishna 6)M. Kanchana 7)O. Manikanta 8)Dr. U. Srinivasa Rao 9)Dr. M. Sunel 10)Dr. Ch. Nagaraju 11)K. Kalpana 12)Bapatla Engineering College (72)Name of Inventor : 1)Dr. Ch.V.M.S.N. Pavan Kumar 2)Dr. Kommalapati Rajesh 3)M. Rupa Mani Latha 4)M. Pavan Krishna 5)M. Kanchana 6)O. Manikanta 7)Dr. P. Surendra Kumar 8)Dr. U. Srinivasa Rao 9)Dr. M. Sunel 10)Dr. Ch. Nagaraju 11)K. Kalpana 12)Bapatla Engineering College
(57) Abstract : Arbitration in a shared resource decides which requester can access a resource when there are multiple requests at the same time. It has a direct impact on the fairness, delay, and power consumption of the system. Conventional Enhanced Ping Lock Arbiters avoid frequent arbitration by holding grants for a fixed time. This method may not be efficient when there are changes in the request patterns, resulting in unnecessary operations and slightly increased delays in light loads. To overcome this limitation, a dynamic group-based Extended Ping Lock Arbiter (EPLA) for four requesters is proposed in this work. In the proposed design, requesters are divided into groups to reduce arbitration activity, and a token is used to select one group at a time. Within the active group, a ping-pong method is applied so that all the requesters within the selected group get a fair chance. After the requester gains access, the time period for which the access will be granted is dynamically varied depending on the number of active requests. If there are no active requests in a group, the skip logic enables the arbiter to skip the group, thus removing unnecessary computations. The fixed priority in the first cycle ensures that the system always starts in the same manner every time. This approach greatly reduces switching activity while maintaining fairness and is appropriate for shared buses and Network-on-Chip architectures No. of Pages : 19 No. of Claims : 4	

Fig: Patent Publication details

Dynamic Group-Based Ping Lock Arbiter with Skip Logic for 4 Requesters

<i>Ch.V.S.M.Pavan Kumar</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(Assistant Professor)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>pavan4790@gmail.com</i>	<i>Kommalapati Rajesh</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(Assistant Professor)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>rajeshkanna2050@gmail.com</i>	<i>M.Rupa Mani Latha</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(U.G.Student)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>rupamanilatha49@gmail.com</i>
<i>M.Pavan Krishna</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(U.G.Student)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>pavanmaruboinao09@gmail.co</i> <i>m</i>	<i>M.Kanchana</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(U.G.Student)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>mirapalakanchana78@gmail.c</i> <i>om</i>	<i>O.Manikanta</i> <i>Dept. of Electronics &</i> <i>Communication Engineering</i> <i>(U.G.Student)</i> <i>Bapatla Engineering College</i> <i>Bapatla,A.P,India</i> <i>ovrlu.7@gmail.co</i>

Abstract: Arbitration in a shared resource decides which requester can access a resource when there are multiple requests at the same time. It has a direct impact on the fairness, delay, and power consumption of the system. Conventional Enhanced Ping Lock Arbiters avoid frequent arbitration by holding grants for a fixed time. This method may not be efficient when there are changes in the request patterns, resulting in unnecessary operations and slightly increased delays in light loads. To overcome this limitation, a dynamic group-based Extended Ping Lock Arbiter (EPLA) for four requesters is proposed in this work. In the proposed design, requesters are divided into groups to reduce arbitration activity, and a token is used to select one group at a time. Within the active group, a ping-pong method is applied so that all the requesters within the selected group get a fair chance. After the requester gains access, the time period for which the access will be granted is dynamically varied depending on the number of active requests. If there are no active requests in a group, the skip logic enables the arbiter to skip the group, thus removing unnecessary computations. The fixed priority in the first cycle ensures that the system always starts in the same manner every time.

This approach greatly reduces switching activity while maintaining fairness and is appropriate for shared buses and Network-on-Chip architectures.

Keywords: Extended Ping Lock Arbiter, EPLA, Group-based Arbitration, Token-based Selection, Ping-Pong Method, Dynamic Grant Lock, Shared Resource Systems, Switching Activity, Fairness.

I. Introduction:

In modern systems like SoCs, NoCs, and FPGAs, there are multiple modules that run in parallel, and all of them try to access the shared resources at the same time. However, only one of the requesters can be served at a time. This is an important aspect of determining the efficiency of the switching activity of the system [1-2].

The arbitration mechanisms, such as fixed priority, round-robin, Least Recently Served (LRS) arbiters, and ping lock arbiters, present many shortcomings. The fixed-priority arbiters can lead to starvation for low-priority requests. The round-robin and LRS mechanisms can yield fairness in favour of higher-priority requests, increasing the internal switching activity because of continuous decision-making [3-4]. The traditional ping lock arbiters can provide a reduction in delay by holding the grants for a constant number of clock cycles. But in the longer version, undesired transitions result in enhanced switching activity [5].

To overcome such issues, this paper presents an EPLA that combines adaptive grant time with an arbitration technique based on groups [6]. Requesters are divided into groups. The token passing technique is used for arbitration between groups to limit unnecessary transitions of grants. In every active group, requesters take turns using an efficient ping-pong technique to give equal chances to all requesters [7-8]. In EPLA, by adapting the grant time to the number of active requesters, switching actions are minimized for both light and heavy traffic, and it performs well from an efficiency perspective without increasing latency [9]. The grouping technique provides better scalability makes it applicable to SoC, NoC, and FPGA setups. Also, simulation results prove that EPLA performs better than the round robin, LRS, and ping-lock arbiter in suppressing SA while providing equal power consumption and delay [10-11]. The rest of this paper is organized as follows. Section II reviews earlier arbitration techniques,

Section IV explains the proposed EPLA architecture, Section V presents simulation results and performance comparison, Section VI concludes the paper, and Section VII provides future work.

II. Related Work

Several arbitration methods have been designed to control access to mutual resources in digital systems. The fixed priority arbiters are quite simple to design and require a prior assignment of priority to all requesters. Requests of the highest priority are served first. Although such arbiters are simpler to design, they may cause fairness issues as lower-priority requesters can starve if higher-priority tasks remain operational continuously [1].

To increase fairness, round-robin arbiters were proposed, wherein the rotational priorities of all the requesters are cyclically increased to give each of them a fair share of the common resource. However, these arbiters have higher internal switching activity due to the updating of the priority and performing arbitration in every clock cycle, resulting in higher dynamic power consumption and longer arbitration delays [2]. These limitations can be overcome using index-based arbitration schemes like Least Recently Served (LRS) arbiters. The LRS arbiters enhance fairness in the design, as the requester who has waited the longest is preferred. Although the above-mentioned arbiters are effective in preventing starvation, the index-based arbiters incur extra complexity in the design as well as an increase in the number of switches due to added comparison logic [3].

The goal of Ping Lock Arbiters is to minimize multiple arbitrations by keeping the grant for multiple clock cycles. As a result, the frequency of making decisions decreases, and the performance becomes better than that of round-robin schemes. Extended ping lock arbiters include additional control logic to control the duration of the grant. However, most of the current designs use a fixed lock duration, which is inefficient in variable traffic. Fixed lock periods usually result in unnecessary switching activity and higher power consumption with limited scalability, as in [4].

However, as the number of requesters rises, many of these arbitration methods are challenged by issues such as efficiency, delay, and scalability. These limitations highlight the need for an arbitration scheme that can be flexible to the traffic pattern

and minimize unnecessary switching while maintaining equal opportunities for all requesters at low hardware cost [5]. Internal switching activity plays a major role in power dissipation for most of the existing arbitration schemes. Round-Robin arbiters depict a moderate dynamic power as frequent pointer updates and grant rotations keep the switching activity high [6]. Round-Robin Scheme-2 optimizes control signals to improve overall on-chip power, but the longer decision cycles increase the data path delay [7]. LRS arbiters, although they reduce the delay, still consume the highest dynamic power since they need to constantly update the priority and make comparisons [8]. Improved Ping Lock arbiters, on the other hand, reduce the number of grant transitions, which reduces static power and data path delay, but maintains moderate dynamic power consumption [9].

The above discussion clearly reveals the need for an arbitration algorithm that can dynamically change the grant time, reduce unnecessary switching, and at the same time provide fair access to various traffic patterns. This brings out the concept of the proposed EPLA, which addresses all the above problems and aims for better scalability and efficiency [10-15].

III. Methodology:

Group Activity Detection

$$G_0 = \begin{cases} 1, & \text{if } Req_0 = 1 \text{ or } Req_1 = 1 \\ 0, & \text{Otherwise} \end{cases}$$

$$G_1 = \begin{cases} 1, & \text{if } Req_2 = 1 \text{ or } Req_3 = 1 \\ 0, & \text{Otherwise} \end{cases}$$

Outer Group Selection

$$Group_{sel} = \begin{cases} 0, & \text{When } Token = 0 \text{ and } G_0 = 1 \\ 1, & \text{When } Token = 1 \text{ and } G_1 = 1 \end{cases}$$

Inner Group Selection

$$Grant_i = \begin{cases} 1, & \text{If the requester } i \text{ is active and selected by the selector} \\ 0, & \text{Otherwise} \end{cases}$$

Request Load Estimation

$$Req_{count} = \sum_{i=0}^3 Req_i$$

Dynamic Grant Lock

$$Dyn_LOCK = \begin{cases} 1, & Req_{count} \leq 1 \\ 2, & Req_{count} = 2 \\ 3, & Req_{count} \geq 3 \end{cases}$$

$$Lock = \begin{cases} 1, & LockCounter > 0 \\ 0, & LockCounter = 0 \end{cases}$$

$$LockCounter_{next} = LockCounter - 1$$

Token Updation

$$Token_{next} = \begin{cases} 1, & \text{if current group inactive and other group active} \\ 0, & \text{if opposite condition holds} \\ Token, & \text{otherwise} \end{cases}$$

Arbitration Delay

$$T_{arb} = T_{group} + T_{request}$$

IV. Proposed work:

The problem with the current Enhanced Ping Lock Arbiter is that it has fixed grant times. The fixed locks reduce the number of arbitration cycles compared to traditional schemes, but they are not effective when the request traffic is varying. If the grant times are varying unnecessarily, the switching activity increases, and the system becomes slow during light loads. To overcome this problem, the Extended Ping-Lock Arbiter (EPLA) is proposed. The EPLA adds dynamic lock management and skip logic. It retains the grants for a longer period during heavy loads to reduce unnecessary arbiters and releases the grants earlier during light loads to improve system responsiveness. Furthermore, the requests are grouped, and the idle groups are skipped to reduce unnecessary work and further reduce power dissipation.

This paper proposes an EPLA that can reduce unnecessary arbitration activity and energy consumption, while maintaining fairness among various requesters. The proposed architecture is shown in Fig. 4.1 and consists of three main blocks: Request

Grouping and Detection Logic, Arbitration Logic, and Grant Encoding with Output Register.

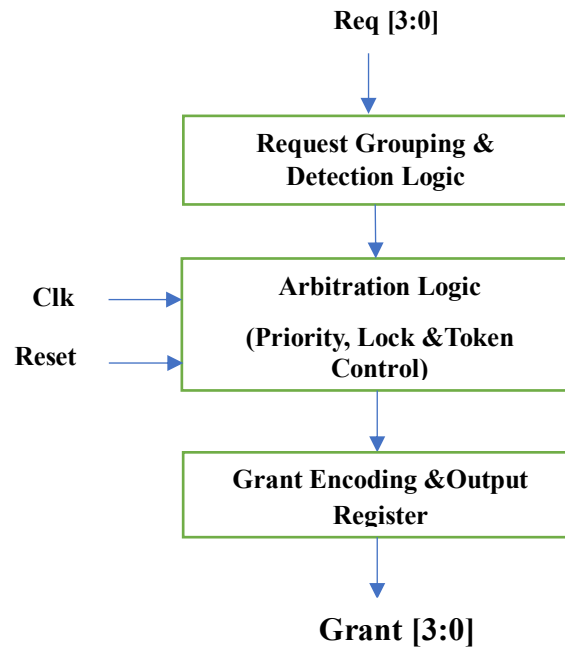


Fig 4.1: Block Diagram of EPLA

The proposed EPLA architecture is represented in Fig. 4.1 and comprises three key blocks: Request Grouping and Detection Logic, Arbitration Logic with Dynamic Lock and Token Control, and finally Grant Encoding with Output Register. Input requests $req[3:0]$ are passed first to the Request Grouping and Detection Logic that groups requesters into predefined groups and detects active groups. Inactive groups are skipped during the arbitration, which helps to save switching activity and power.

This arbitration logic operates synchronously with a system clock Clk and a reset signal $reset$. It integrates the function of priority control with token-based group selection and a dynamic grant lock mechanism. A rotating token ensures fairness in selecting groups and avoids starvation. In each selected group, it fairly selects one active requester using a ping-pong pointer mechanism. Unlike in designs for fixed-duration ping lock, the duration of a grant lock is dynamically adjusted according to the number of active requests, with a longer lock for heavy load and a shorter lock for light load. Such a

dynamic lock strategy greatly reduces useless repeated arbitration operations, hence reducing switching activities and power consumption.

The Grant Encoding and Output Register module uses the results of arbitration to produce the final grant signals Gnt [3:0]. The grant signals are encoded and stored in order to synchronize communication between the arbiter and the shared resource so that only one requester gets access simultaneously. Although dynamic lock control has very little impact on increased control activities, skip logic eliminates unnecessary arbiters to save power consumption.

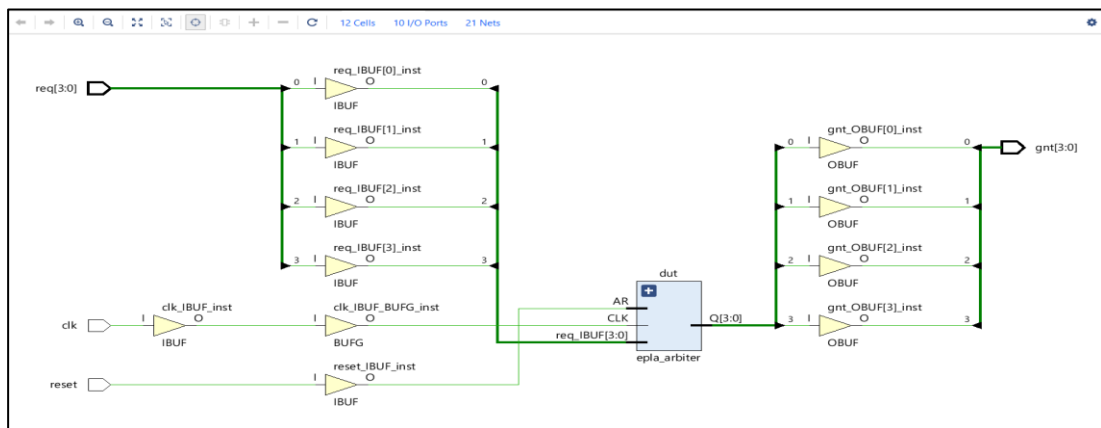


Fig 4.2: Implementation Schematic Diagram of EPLA

The implementation schematic of the proposed EPLA is represented in Fig 4.2. The clock is then forwarded to input as well as global buffers to enable synchronization. The request inputs Req [3:0], as well as the reset signal, are connected to input buffers to ensure that they are received by the arbiter core correctly. The EPLA module and the resulting grant signals Gnt[3:0] are connected to output buffers before being forwarded to the output ports. At a time, only one grant signal is asserted to ensure proper arbitration and proper allocation of resources in the FPGA module.

This work was designed by selecting an Xilinx Artix-7 FPGA device and implemented on the Nexys-4 DDR FPGA board using user-defined constraints represented below in Fig.4.3. System clock input was fed manually through the center push button BTNC (pin N17) in order to provide controlled clock pulses during hardware verification. The reset signal is reset-mapped to switch SW0 (pin J15), thus allowing proper initialization

of the arbiter. The request inputs request[3:0] were assigned to onboard switches SW1–SW4 such that different combinations of requests can be applied in real time.

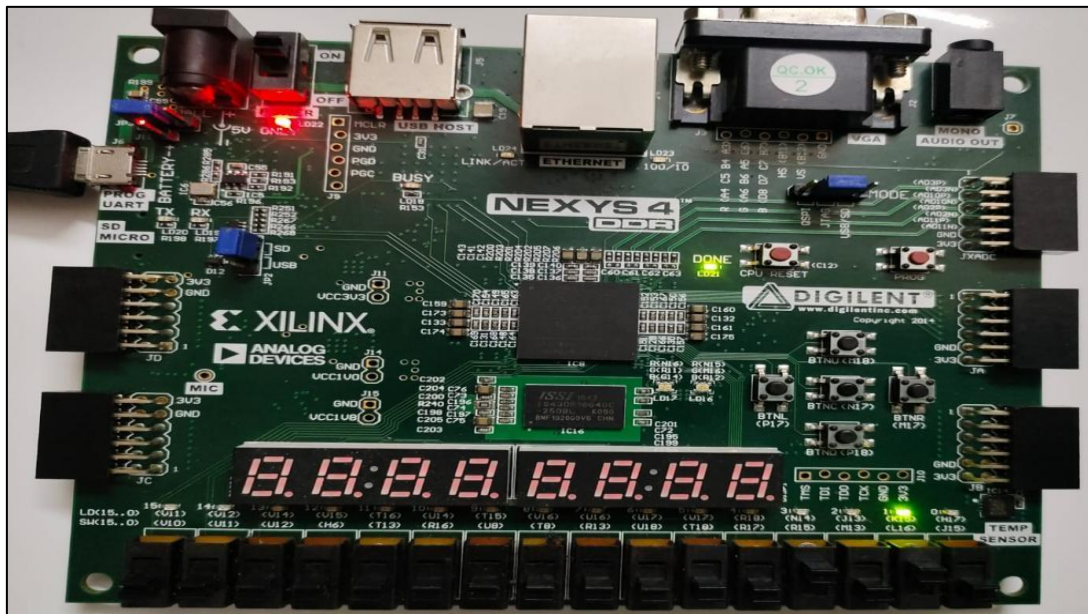


Fig 4.3: FPGA Implementation

The grant outputs [3:0] had been mapped to LEDs, LED1–LED4, such that the arbitration result will be directly observable visually. During operation, with each clock button press, the arbitration moves one cycle forward, and only one LED is seen to glow at any instant, confirming mutual exclusion. In addition, if many requests are turned on at the same time, the glowing LED changes serially with each click of the clock button, thereby showing fair rotation of grants. This hardware behaviour gives quite similar results as obtained during simulation, thus justifying the correctness of the proposed EPLA design.

V. Results:

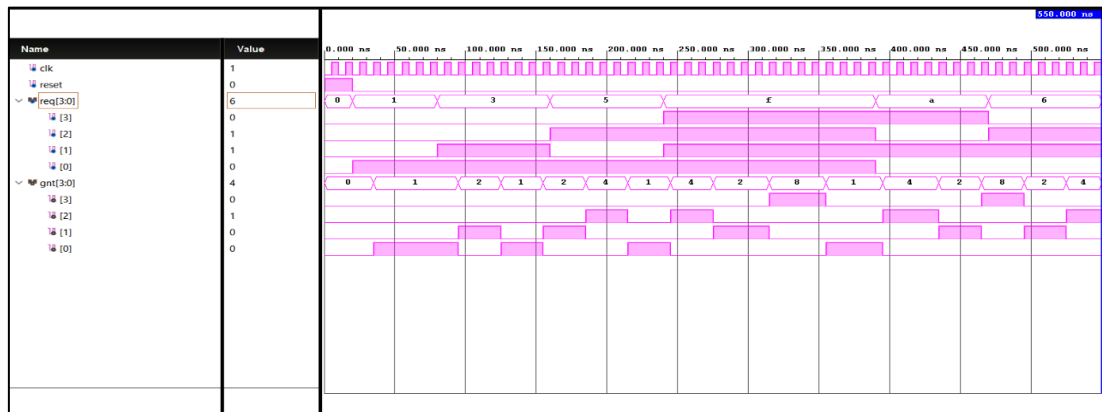


Fig 5.1 Simulation Results of EPLA

From the waveform of the simulation above Fig 5.1: represents, under no request condition (request = 0000), there is no grant, and the lock remains inactive. For a single active request (request = 0001), the lock is asserted, and its grant is generated (grant = 0001) with the smallest lock period (lock count = 0), ensuring no unnecessary switching occurs. For the situation where two requests are asserted (request = 0011), the grant is issued to one of the requesters (grant = 0001), which is held with a dynamic lock (lock count = 1). After the lock expires, it is transferred to the grant of another requesting entity (grant= 0010), switching repeatedly in a controlled manner. For the remaining two request combinations of request = 0101 and request = 1010, under similar conditions, it is observed that the grant for the requests rotates among the requesters who are active, skipping the inactive requesters. In the fully loaded condition of request = 1111, the lock period is set to its largest value of lock count = 2, and the grant rotation takes place sequentially among all requesters, with grant rotating from 0001, 0100, 0010 and 1000, respectively, with each grant maintained for several clock cycles. When the number of active requests decreases (e.g., request = 0110), the lock period also decreases, and the grant transitions are restricted among the active requesters. The behavior described here, revealed from the waveform, supports the fact that the proposed arbiter is capable of dynamically changing the lock period with respect to the active number of requests to limit the grant transitions.

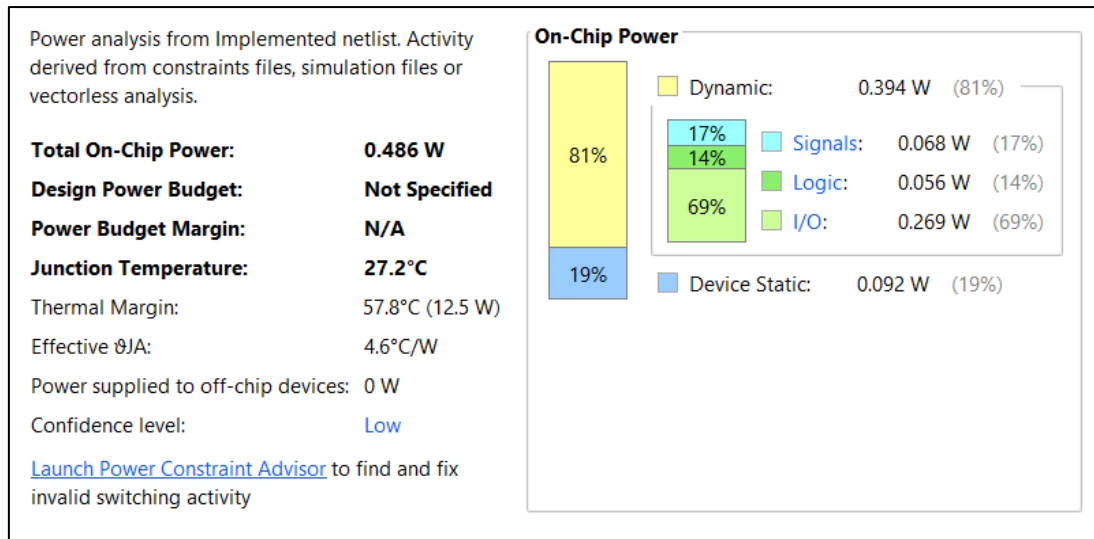


Fig 5.2: Power Analysis

From Fig 5.2 the power analysis of the integrated design of EPLA shows that there is a total power consumption of 0.486 W. Dynamic power contributes predominantly to this power consumption of 0.394 W (81%), along with static power of 0.092 W (19%) associated with devices. Within dynamic power, I/O power dominates other powers by contributing a maximum of 0.269 W to power consumption (69%), followed by signal power of 0.068 W (17%) and logic power of 0.056 W (14%). The power analysis indicates that the total power consumption is highly dependent on FPGA I/O activity, while the internal logic and signal power remain relatively low due to reduced arbitration operations.

Timing Report

Slack: inf
Source: dut/gnt_reg[0]/C
(rising edge-triggered cell FDCE)
Destination: gnt[0]
(output port)
Path Group: (none)
Path Type: Max at Slow Process Corner
Data Path Delay: 4.837ns (logic 3.104ns (64.182%) route 1.732ns (35.818%))
Logic Levels: 2 (FDCE=1 OBUF=1)

Location	Delay type	Incr(ns)	Path(ns)	Netlist Resource(s)
SLICE_X0Y53	FDCE	0.000	0.000	r dut/gnt_reg[0]/C
SLICE_X0Y53	FDCE (Prop_fdce_C_Q)	0.456	0.456	r dut/gnt_reg[0]/Q
	net (fo=1, routed)	1.732	2.188	gnt_OBUF[0]
R10	OBUF (Prop_obuf_I_O)	2.648	4.837	r gnt_OBUF[0]_inst/O
	net (fo=0)	0.000	4.837	gnt[0]
R10				r gnt[0] (OUT)

Fig 5.3: Timing Report

The results for timing analysis from Fig.5.3 reveal that the designed system satisfies all the timing constraints with positive slack values. The longest delay path for data transfer is 4.837ns with logic and routing contributions of 64.18% and 35.82%, respectively. This is due to the propagation path that involves only two levels of logic for the data to move from the grant register to the output.

Table 5: Performance Comparison

S. No	Comparison	Dynamic Power (W)	Total ON-Chip Power (W)	Switching Activity
1	Proposed Extended Ping Lock Arbiter	0.394	0.486	Reduced
2	Existing (Enhanced ping lock arbiter)	1.089	1.172	High
3	% change w.r.t Existing	63.8%	58.5%	Significantly Reduced

VI. Conclusion

The proposed adaptive lock control dynamically adjusted the grant time according to the activity in the requesters with dynamic lock controlling, grouping, and skip logic to improve the efficiency of the arbiter in systems with shared resources. Grouping with skip logic relieved the arbiter from the processing of inactive requesters. As a result, switching activity was reduced by 63.8%. Thus, it has been established that the proposed arbiter resulted in a remarkable reduction in internal transitions compared to the enhanced ping lock arbiter and the total on-Chip power was reduced by 58.5% from the existing system. The proposed arbiter has been found appropriate for efficient and scalable resource sharing in VLSI and SoC designs.

VII. Future Work

Further extensions could be scaling the EPLA to support more requesters and multiple hierarchical groups for complex SoC or NoC systems. The token rotation and dynamic lock mechanism could be further developed for more adaptive arbitration, while the integration with automated high-speed clocks may provide better responsiveness in real-time applications.

References

- [1] P. Karanpuria, "Fair chance round robin arbiter," *Int. J. Comput. Appl.*, vol. 81, no. 14, pp. 36–42, Nov. 2013.
- [2] R. Shashidhar, S. N. Sujay, and G. S. Pavan, "Implementation of bus arbiter using round robin scheme," *Int. J. Innov. Res. Sci., Eng. Technol.*, vol. 3, no. 7, pp. 14937–14944, Jul. 2014.
- [3] A. S. N. Asha and B. A. U. Bindu, "Design and implementation of index-based round robin arbiter for NoC routers using FPGA," *Int. J. Innov. Res. Sci., Eng. Technol.*, vol. 5, no. 5, pp. 7448–7453, May 2016, doi: 10.15680/IJIRSET.2016.0505141.
- [4] A. Mishra, "Design of a round robin bus arbiter using SystemVerilog," *Int. Res. J. Eng. Technol. (IRJET)*, vol. 7, no. 1, pp. 1492–1494, Jan. 2020.
- [5] N. L. Venkataraman et al., "Design of matrix, distributive round robin, ping pong and enhanced ping lock arbiter for shared resources systems," *Indones. J. Electr. Eng. Comput. Sci.*, vol. 32, no. 3, pp. 1337–1345, Dec. 2023, doi: 10.11591/ijeecs.v32.i3.pp1337-1345.
- [6] N. L. Venkataraman et al., "Design and implementation of different arbiters for shared resource systems," *Theory Appl. Eng. Res.*, vol. 4, 2023, doi: 10.9734/bpi/taer/v4/2779G.
- [7] N. M. and V. Shylaja, "FPGA implementation of priority-arbiter based router design for NoC systems," *Int. J. Adv. Res. Eng. Technol.*, vol. 10, no. 2, pp. 509–516, Mar.–Apr. 2019.

- [8] T. Kavitha, S. Shiyamala, and P. Nagarajan, "FPGA implementation of arbiters algorithm for network-on-chip routers," *ARPJ. Eng. Appl. Sci.*, vol. 11, no. 19, pp. 11451–11456, 2016.
- [9] M. Abdelrasoul, M. Ragab, and V. Goulart, "Evaluation of the scalability of round robin arbiters for NoC routers on FPGA," in *Proc. IEEE MCSoc*, 2013.
- [10] B. Debbarma and J. Debbarma, "Clock-gated round robin arbiter for NoC router," *MATTER: Int. J. Sci. Technol.*, vol. 3, no. 3, pp. 113–124, Nov. 2017.
- [11] R. S. Shashidhar and G. S. Pavan, "FPGA implementation of a bus arbiter for shared resources," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 5, no. 6, pp. 123–128, Jun. 2016.
- [12] N. M. and V. Shylaja, "Design and implementation of priority arbiter based router," *Int. J. Adv. Res. Eng. Technol.*, vol. 10, no. 2, pp. 509–516, 2019.
- [13] T. Kavitha, S. Shiyamala, and P. Nagarajan, "FPGA implementation of arbiters algorithm for network-on-chip routers," *ARPJ. Eng. Appl. Sci.*, vol. 11, no. 19, pp. 11451–11456, 2016.
- [14] A. S. Tigadi and H. Guhilot, "Design of an arbiter for two systems accessing a single DDR3 memory on a reconfigurable platform," *Int. J. Inf. Eng. Electron. Bus.*, vol. 10, no. 6, pp. 14–20, Nov. 2018, doi: 10.5815/ijeeb.2018.06.02.
- [15] A. Paul et al., "Reconfigurable parallel architecture of high-speed round robin arbiter," *arXiv preprint*, Mar. 2024
- [16] Adrika Mohanty and Chiranjeevi G N., "Comparative Analysis of Round Robin Arbiter and Weighted Round Robin Arbiter," pp. 26-27 July 2024, Publisher: IEEE, DOI: 10.1109/APCIT62007.2024.10673654.
- [17] Anuradha Shenoy, Anika Kirana Venkat, Adithya S. Chakravarty, Smitha Gayathri D. "Implementation of Fair Chance Round Robin Arbiter and its Comparative Analysis," pp: 26-27 September 2025, Publisher: IEEE, DOI: 10.1109/ICWITE64848.2025.11307144.

- [18] K. Yoghigoe, K.J. Christensen, A. Roginsky, “Design of a high-speed overlapped round robin (ORR) arbiter,” pp: 20-24 October 2003, Publisher: IEEE, DOI: 10.1109/LCN.2003.1243193.
- [19] Khaled A. Helal, Sameh Attia, Tawfik Ismail, Hassan Mostafa, “Priority-select arbiter: An efficient round-robin arbiter,” pp: 07-10 June 2015, DOI: 10.1109/NEWCAS.2015.7182062.
- [20] J. Reed, N. Manjikian, “A dual round-robin arbiter for split-transaction buses in system-on-chip implementations,” pp: 2004, IEEE Cat No. 04CH37513)
- [21] S. Q. Zheng, Mei Yang, J. Blanton, P. Golla, D. Verchere “A simple and fast parallel round-robin arbiter for high-speed switch control and scheduling,” pp: 2002, DOI: 10.1109/MWSCAS.2002.1186951.
- [22] Jagrati Gupta, Nidhi Goel, “Efficient bus arbitration protocol for SoC Design,” pp: 2015, Publisher: IEEE
- [23] D. Shanthi, R. Amutha, “Design of efficient on-chip communication architecture in MpSoC,” pp: 2006, Publisher: IEEE, DOI: 10.1109/ICNICONSMCL.2006.168
- [24] Abhilash L Bedre, V. Nithish Kumar, “A Hybrid arbiter to accelerate performance of high speed SoC,” pp: 2017, Publisher: IEEE
- [25] Subhah Gogula, V. Damodaran, “Design of a VLSI Router For the faster Data Transmission Using Buffer,” pp: 21-22 April 2023, Publisher: IEEE, DOI: 10.1109/ICSTSN57873.2023.101568.

